

锐能微 SOC_MCU 应用笔记 018

SEA 加解密库函数使用说明

Rev1.0.6

目录

1. 算法认证	4
2. 版本更新说明	5
3. AES 库函数	6
3.1. 函数接口	6
3.2. 函数说明	6
3.2.1. 初始化函数	6
3.2.2. 加密、签名函数	8
3.2.3. 解密、认证函数	8
3.2.4. GMAC 加解密函数	9
3.2.5. 密钥封装函数	9
3.2.6. 密钥解封装函数	10
3.2.7. AAD 传入函数	10
3.3. 验证函数	10
3.3.1. 加解密函数 Demo	10
3.3.2. GMAC 函数 Demo	16
3.3.3. 密钥封装和解封装 Demo	20
3.3.4. AAD 多次传入 demo	24
3.3.5. 过程现场保存和恢复	36
4. HASH 库函数	38
4.1. 函数接口	38
4.2. 函数说明	38
4.2.1. 初始化函数	38
4.2.2. 计算函数	38
4.3. 验证函数	39
4.3.1. 伪代码 Demo	39
4.3.2. HASH Demo	40
4.3.3. 过程现场保存和恢复	47
5. ECC 库函数	49
5.1. 函数接口	49
5.2. 函数说明	49
5.2.1. 公私密钥对生成函数	49
5.2.2. 公钥验证函数	50
5.2.3. ECDSA 签名生成函数 I	50
5.2.4. ECDSA 签名验证函数	51
5.2.5. ECDSA 生成签名函数 II	52
5.2.6. 公钥生成函数	52
5.2.7. 密钥协商函数	53
5.2.8. 带有非压缩编码格式的公钥生成函数	53
5.3. 验证函数	54
5.3.1. 自签名自校验函数	54
5.3.2. NIST 标准数据验证函数	55

6. RSA 库函数	56
6.1. 函数接口	56
6.2. 函数说明	56
6.2.1. 公私钥对生成函数	56
6.2.2. 签名生成函数	57
6.2.3. 签名验证函数	58
6.2.4. 加解密函数	58
6.3. 验证函数	60
6.3.1. 密钥生成 Demo	60
6.3.2. 加解密 Demo	60
6.3.3. 签名和验签 Demo	62
7. 其他功能	64
7.1. 可选宏定义	64
7.1.1. AES 地址对齐方式定义	64
7.1.2. HASH 地址对齐方式定义	64
7.1.3. ECC 地址对齐方式定义	64

1. 算法认证

安全密码加速器 (Security Encryption Accelerator) 是为对称加密算法 (AES)、公钥密码算法 (ECDSA、ECDH 等) 提供硬件加速的模块。包含 AES 硬件加速单元、PKA 公钥算法加速器 (ECC 硬件加速单元、RSA 硬件加速单元)、散列算法加速器、128 位有限域乘法单元、真随机数发生器。

以本模块为基础的固件 RS-SEA 通过 FIPS140-3 算法认证。

认证详情请参见 NIST 官网: [Cryptographic Algorithm Validation Program | CSRC \(nist.gov\)](https://csrc.nist.gov/Cryptographic-Algorithm-Validation-Program)

2. 版本更新说明

版本号	时间	说明
v1.0.0	2023-02	AES: 1. 支持 ECB/CBC/CFB128/OFB/CRT 数据加密、解密; 2. 支持 GCM 密钥传输方案; 3. 支持 GAMC 消息鉴别码算法; 4. 支持 AES-128/AES-192/AES-256 长度密钥。 HASH: 1. 支持 SHA1/SHA224/SHA256/SHA384/SHA512; 2. 支持任意数据长度运算; 3. 支持多次拼接运算。 ECC: 1. 支持 ECC 公私密钥对生成、公钥验证; 2. 支持 ECDSA 签名生成、签名验证。
v1.0.1	2023-05	AES 新增支持: 1. 增加明文、密文、IV 数据、附件消息 U8 地址对齐支持宏定义配置项; 2. 增加过程保存和恢复功能; 3. 支持对 ADD 和密文数据进行单独 GMAC 计算; 4. 将 key、tag 地址指针修改为 U8 地址对齐。 HASH 新增支持: 1. 支持多次拼接运算, 过程数据长度支持任意长度; 2. 支持过程现场保存和恢复功能。用于多于 2 种数据类型的数据进行交叉式 HASH 计算。
v1.0.2	2024-11	新增支持 RSA 加解密运算: 1. 数据位宽支持 RSA-512/RSA-1024; 2. 支持 PKCS#1 V1.5 填充方案的 RSA 签名和验签; 3. 支持公钥格式为 (N, E) 和私钥格式为 (N, D) 的 RSA 加解密; 4. 仅支持 512bits 的 RSA 公私钥对生成。
v1.0.3	2025-03	ECC 新增支持: 1. 新增支持 ECC Brainpool P-256/384/512 曲线计算。
v1.0.4	2025-03	AES 新增支持: 1. 新增支持 AES KeyWrap 密钥封装。
v1.0.5	2025-04	AES 新增支持: 1. 新增支持 AES 可多次传入 AAD 附加消息。 ECC 新增支持: 1. 新增支持 ECC 密钥协商输出 X 坐标; 2. 新增支持 ECC 公钥生成输出带有编码格式的公钥对。
v1.0.6	2025-04	ECC 新增支持: 1. 新增支持 ECC 地址字节对齐。

3. AES 库函数

3.1. 函数接口

文件:

- rn8xxx_ll_sea_aes.c
- rn8xxx_ll_sea_aes.h

函数名	函数接口名
加解密初始化函数	ErrStatus LL_AES_crypt_init(const aesCypherConfigStr* configuration);
AES 模块关闭函数	ErrStatus LL_AES_crypt_deinit(void);
加密函数、加密和签名函数	ErrStatus LL_AES_crypt_sign(const plainData_t* plainData, uint32_t size, CYPHER_DATA_STA is_last, cryptData_t* cryptData);
解密函数、解密和认证函数	ErrStatus LL_AES_decrypt_verify(const cryptData_t* cryptData, uint32_t size, CYPHER_DATA_STA is_last, plainData_t* plainData);
GMAC 签名、认证函数	ErrStatus LL_AES_gmac(const ghashData_t* ghashData, uint32_t size, CYPHER_DATA_STA is_last);
加解密过程现场保存函数（扩展功能，非必须）	ErrStatus LL_AES_process_status_latch(aesProcessParameters* latch_buf);
加解密过程现场恢复函数（扩展功能，非必须）	ErrStatus LL_AES_process_status_recover(const aesProcessParameters* recover_buf);
密钥封装函数	ErrStatus LL_AES_key_wrap(const plainData_t *plainData, uint32_t in_len, cryptData_t *cryptData, uint32_t *out_len);
密钥解封装函数	ErrStatus LL_AES_key_unwrap(const cryptData_t *cryptData, uint32_t in_len, plainData_t *plainData, uint32_t *out_len);
AAD 输入函数	ErrStatus LL_AES_update_aad(const add_t *aadData, uint32_t size);

3.2. 函数说明

3.2.1. 初始化函数

ErrStatus LL_AES_crypt_init(const aesCypherConfigStr* configuration);

加密、解密、GMAC 使用同一个初始化函数。

3.2.1.1. configure 结构体

```
typedef struct {
    AES_ENCRYPT_SEL        encrypt_sel;
    AES_BITWIDE_MODE      bitWideMode;
    AES_CRYPT_MODE        cryptMode;
    key_t*                keyLocation;
    uint32_t               keySize;
    iv_t*                  ivLocation;
    uint32_t               ivSize;
    add_t*                 addLocation;
    uint32_t               addSize;
    tag_t*                 tagSignBuf;
}
```

```
uint32_t      tagSize;
tag_t*        tagVerifyBuf;
AES_VERIFY_STA* tagVerifySta;
AES_SPECIAL_MOD specialMode; //v1.0.1 新增
} aesCypherConfigStr;
```

3.2.1.2. configure 参数描述

参数	说明	适用对象	注意事项
encrypt_sel	加密/解密初始化选择	全部	AES_enCrypt/AES_deCrypt
bitWideMode	AES Key 长度选择	全部	AES_128/AES_192/AES_256
cryptMode	数据流处理模式选择	全部	AES_ECB/CBC/CTR/CFB128/OFB/GCM
keyLocation	指针指向密钥地址	全部	指针指向地址 byte 对齐； 数据存储方式为 Little-Endian。
keySize	密钥长度,函数内部根据 bitWideMode 产生，无需配置	NC	最新版本中该参数无需配置。
ivLocation	指针指向初始向量地址	ECB 除外	指针指向地址 byte/word 对齐； 数据存储方式为 Little-Endian。
ivSize	初始向量长度	GCM	单位规定为 bytes。
addLocation	指针指向附加消息地址	GCM	指针指向地址 byte/word 对齐； 数据存储方式为 Little-Endian。
addSize	附加消息长度	GCM	单位规定为 bytes。
tagSignBuf	加密：指针指向签名输出结果存放的 buffer 地址； 解密：指向用于认证的签名的输入地址	GCM	指针指向地址 byte 对齐； 数据存储方式为 Little-Endian。 buffer size>=16 bytes。
tagSize	加密：输出数字签名长度； 解密：认证数字签名长度。	GCM	单位规定为 bytes，tagSize<=16 bytes。
tagVerifyBuf	解密：指针指向认证输出结果存放的 buffer 地址。	GCM	指针指向地址 byte 对齐； 数据存储方式为 Little-Endian。 buffer size>=16bytes。
tagVerifySta	解密：指针指向认证输出结果存放的地址。	GCM	返回结果： ◆ =0：认证成功，tagSign = tagVerify； ◆ =1：认证失败。
specialMode	特殊模式配置参数。 aes_gmac 函数： 1. specialMode = AES_SPC_NONE，输入数据为 ADD 数据。 2. specialMode = AES_SPC_GMAC_CryptData，输入数据为密文数据	全部	正常应用请配置为 0。 特殊应用详看 specialMode 参数说明。

3.2.1.3. specialMode 参数说明

此参数用于配置加解密函数的特殊模式使用，配合加解密函数使用。默认情况下，请配置此参数 = 0

(AES_SPC_NONE)。

3.2.1.3.1. aes_gmac 函数应用

作用对象： aes_gmac 函数应用，输入数据 ghashData 类型

配置操作：

- ghashData 为 ADD 数据：specialMode = AES_SPC_NONE
- ghashData 为密文数据：specialMode = AES_SPC_GMAC_CryptData

3.2.2. 加密、签名函数

ErrStatus LL_AES_crypt_sign(const plainData_t* plainData, uint32_t size, CYPHER_DATA_STA is_last, cryptData_t* cryptData);

3.2.2.1. 接口参数描述

参数	说明	适用对象	注意事项
plainData	指针指向明文地址	全部	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
size	明文长度	全部	单位规定为 bytes。
is_last	加密数据状态配置	GCM	当计算为完整数据的最后一次计算时，需要将 is_last 置 1，其他情况置 0。 is_last = 1，计算 GCM 完成后，将在 tagSignBuf 里面输出签名信息 TAG。
cryptData	指针指向密文输出存放的 buffer 地址	GCM	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。

3.2.2.2. is_last 参数说明

1. 当 is_last = 1 的时候，表示完成全部明文的加密，并可以进行下一组明文加密。如果此时明文 size 不为 16（byte）的整数倍，则会补零。在 GCM 模式下，同时在 tagSignBuf 输出 tag；
2. 当 is_last = 0 时候，必须保持明文 size 等于 16bytes 的整数倍。

3.2.3. 解密、认证函数

ErrStatus LL_AES_decrypt_verify(const cryptData_t* cryptData, uint32_t size, CYPHER_DATA_STA is_last, plainData_t* plainData);

3.2.3.1. 接口参数描述

参数	说明	适用对象	注意事项
cryptData	指针指向密文地址	全部	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
size	明文长度	全部	单位规定为 bytes。
is_last	加密数据状态配置	GCM	此次 GCM 计算为完整数据的最后一次计算时，需要将 is_last 置 1，其他情况置 0。

			当 is_last = 1，将在解密完成后在 tagVerifyBuf 里面输出签名信息 TAG，和认证结果 tagVerifySta。
plainData	指针指向解密后明文输出存放的 buffer 地址	GCM	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。 buffer size>=size。

3.2.3.2. is_last 参数说明

1. 当 is_last = 1 的时候，表示完成全部密文的解密，并可以进行下一组密文解密。如果此时明文 size 不为 16（byte）的整数倍，则会补零。在 GCM 模式下，在 tagVerifyBuf 输出计算出来的 tag，同时输出认证结果 tagVerifySta；
2. 当 is_last = 0 时候，必须保持明文 size 等于 16bytes 的整数倍。

3.2.4. GMAC 加解密函数

ErrStatus LL_AES_gmac(const ghashData_t* ghashData, uint32_t size, CYPHER_DATA_STA is_last);

使用条件：cryptMode=GCM。

3.2.4.1. 接口参数 ghashData 描述

参数	说明	适用对象	注意事项
ghashData	指针指向明文地址	全部	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
size	明文长度	全部	单位规定为 bytes。
is_last	加密数据状态配置	GCM	当计算为完整数据的最后一次计算时，需要将 is_last 置 1，其他情况置 0。 is_last = 1，计算 GMAC 完成后，将在 tagSignBuf 里面输出签名信息 TAG。

3.2.4.2. ghashData 参数说明

1. specialMode = AES_SPC_GMAC_CryptData，ghashData 参数为密文；
2. specialMode = 0/其他值，ghashData 参数为附件消息（ADD）。

3.2.4.3. is_last 参数

1. 当 is_last = 0 时候，必须保持明文 size 等于 16bytes 的整数倍；
2. 当 is_last = 1 的时候，表示完成一帧数据全部 GMAC 计算，可以继续下一帧数据的计算。
 - (1) 解密模式下，在 tagSignBuf 输出 tag；
 - (2) 解密模式下，在 tagVerifyBuf 输出计算出来的 tag，同时输出认证结果 tagVerifySta。

3.2.5. 密钥封装函数

ErrStatus LL_AES_key_wrap(const plainData_t *plainData, uint32_t in_len, cryptData_t *cryptData, uint32_t *out_len);

密钥封装函数仅适用于 AES_ECB 计算。

3.2.5.1. 接口参数描述

参数	说明	适用对象	注意事项
plainData	指针指向需要封装的密钥地址	ECB	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
in_len	需要封装的密钥长度	ECB	单位规定为 bytes。
crypteData	指针指向密钥封装后输出存放的 buffer 地址	ECB	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
out_len	密文输出长度	ECB	单位规定为 bytes。

3.2.6. 密钥解封装函数

ErrStatus LL_AES_key_unwrap(const cryptData_t *crypteData, uint32_t in_len, plainData_t *plainData, uint32_t *out_len);

密钥解封装函数仅适用于 AES_ECB 计算。

3.2.6.1. 接口参数描述

参数	说明	适用对象	注意事项
crypteData	指针指向密文地址	ECB	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
in_len	密文长度	ECB	单位规定为 bytes。
plainData	指针指向解封装后密钥输出存放的 buffer 地址	ECB	指针指向地址 bytes 对齐；数据存储方式为 Little-Endian。
out_len	明文输出长度	ECB	单位规定为 bytes。

3.2.7. AAD 传入函数

ErrStatus LL_AES_update_aad(const add_t *aadData, uint32_t size);

适用于 AES_GCM 和 GMAC 计算。

3.2.7.1. 接口参数描述

参数	说明	适用对象	注意事项
aadData	指针指向附加消息地址	GCM	指针指向地址 bytes 对齐。
size	附加消息长度	GCM	单位规定为 bytes。

3.3. 验证函数

3.3.1. 加解密函数 Demo

说明：函数使用的时候注意判定函数 return 值，当 return 值=SUCCESS 的时候，函数操作和数据才有效。

3.3.1.1. GCM 典型数据

3.3.1.1.1. 文本数据（字符串）

- 标准字符串输入数据(文本数据，单纯参考使用)
"key": "feffe9928665731c6d6a8f9467308308"
"pt": "d9313225f88406e5a55909c5aff5269a86a7a9531534f7da2e4c303d8a318a721c3c0c95956809532fcf0e2449a6b525b16aedf5aa0de657ba637b39"
"add": "feedfacedeadbeeffeedfacedeadbeefabaddad2"
"iv": "cafebabefacedbaddecaf888"
- 标准字符串输出数据(文本数据)
"ct": "42831ec2217774244b7221b784d0d49ce3aa212f2c02a4e035c17e2329aca12e21d514b25466931c7d8f6a5aac84aa051ba30b396a0aac973d58e091"
"tag": "5bc94fbc3221a5db94fae95ae7121a47"

3.3.1.1.2. 单片机内存数据（16 进制小端）

```
uint8_t aes_128_gcm_key_test0[] = {0xfe,0xff,0xe9,0x92,0x86,0x65,0x73,0x1c,0x6d,0x6a,0x8f,0x94,0x67,0x30,0x83,0x8};  
uint8_t aes_128_gcm_iv_test0[] = {0xca,0xfe,0xba,0xbe,0xfa,0xce,0xdb,0xad,0xde,0xca,0xf8,0x88};  
uint8_t aes_128_gcm_tag_test0[] = {0x5b,0xc9,0x4f,0xbc,0x32,0x21,0xa5,0xdb,0x94,0xfa,0xe9,0x5a,0xe7,0x12,0x1a,0x47};  
uint8_t aes_128_gcm_pt_test0[] = {0xd9,0x31,0x32,0x25,0xf8,0x84,0x06,0xe5,0xa5,0x59,0x09,0xc5,0xaf,0xf5,0x26,0x9a,0x86,0xa7,0xa9,0x53,0x15,0x34,0xf7,0xda,0x2e,0x4c,0x30,0x3d,0x8a,0x31,0x8a,0x72,0x1c,0x3c,0x0c,0x95,0x95,0x68,0x09,0x53,0x2f,0xcf,0x0e,0x24,0x49,0xa6,0xb5,0x25,0xb1,0x6a,0xed,0xf5,0xaa,0x0d,0xe6,0x57,0xba,0x63,0x7b,0x39};  
uint8_t aes_128_gcm_add_test0[] = {0xfe,0xed,0xfa,0xce,0xde,0xad,0xbe,0xef,0xfe,0xed,0xfa,0xce,0xde,0xad,0xbe,0xef,0xab,0xad,0xda,0xd2};  
uint8_t aes_128_gcm_ct_test0[] = {0x42,0x83,0x1e,0xc2,0x21,0x77,0x74,0x24,0x4b,0x72,0x21,0xb7,0x84,0xd0,0xd4,0x9c,0xe3,0xaa,0x21,0x2f,0x2c,0x02,0xa4,0xe0,0x35,0xc1,0x7e,0x23,0x29,0xac,0xa1,0x2e,0x21,0xd5,0x14,0xb2,0x54,0x66,0x93,0x1c,0x7d,0x8f,0x6a,0x5a,0xac,0x84,0xaa,0x05,0x1b,0xa3,0x0b,0x39,0x6a,0x0a,0xac,0x97,0x3d,0x58,0xe0,0x91};
```

3.3.1.1.3. 变量定义

```
uint8_t tag_sign[16];  
uint8_t tag_verify[16];  
uint32_t crpyt_decrypt_data[128];  
AES_VERIFY_STA verify_status;
```

说明：

- 加密：tagSignBuf 会在最后一组数据，is_last = 1 的时候返回 tag 计算结果；

- 解密：verify_status, tag_verify 会在最后一组数据 (is_last = 1) 的时候返回 tag 计算结果和校验标志。

3.3.1.1.4. 变量初始化

```
void aes_128_gcm_demo_init(AES_ENCRYPT_SEL encrypt_sel)
{
    ErrStatus ret_error ;
    aesCypherConfigStr aes_gcm_cfg ;
    // 1.配置参数初始化
    aes_gcm_cfg.encrypt_sel      = encrypt_sel;
    aes_gcm_cfg.bitWideMode     = AES_128;
    aes_gcm_cfg.cryptMode       = AES_GCM;
    aes_gcm_cfg.keyLocation     = (key_t *)aes_128_gcm_key_test0;
    aes_gcm_cfg.keySize         = 0;//RESERVED
    aes_gcm_cfg.ivLocation      = (iv_t *)aes_128_gcm_iv_test0;
    aes_gcm_cfg.ivSize          = sizeof(aes_128_gcm_iv_test0) ;
    aes_gcm_cfg.addLocation     = (add_t *)aes_128_gcm_add_test0;
    aes_gcm_cfg.addSize         = sizeof(aes_128_gcm_add_test0) ;
    aes_gcm_cfg.tagSignBuf      = (tag_t *)tag_sign;
    aes_gcm_cfg.tagVerifyBuf    = (tag_t *)tag_verify;
    aes_gcm_cfg.tagSize         = sizeof(aes_128_gcm_tag_test0) ;
    aes_gcm_cfg.tagVerifySta    = &verify_status;
    aes_gcm_cfg.specialMode     = AES_SPC_NONE; /*v1.0.1 新增*/
    // 2.初始化
    ret_error = aes_crypt_init((const aesCypherConfigStr*) &aes_gcm_cfg);
    if(ret_error != SUCCESS){
        printf("\naes_crypt_init return error\n");
        return;
    };
}
```

3.3.1.2. 单次完成加密解密

可以支持任意数据长度的单次加密。

3.3.1.2.1. 加密和签名

```
void aes_128_gcm_demo_crypt_sign(void)
{
    ErrStatus ret_error ;
    // 1.加密和签名
    ret_error = aes_crypt_sign((const plainData_t*)(aes_128_gcm_pt_test0)
    , sizeof(aes_128_gcm_pt_test0), CypherDataIsTheLast
    , (cryptData_t*)(crpyt_decrypt_data));
    if(ret_error != SUCCESS){
        printf("\naes_crypt_sign return error\n");
        return;
    };
}
```

```
};
// 2.比对加密结果-密文
if(0 == memcmp((const uint8_t*)aes_128_gcm_ct_test0
, (const uint8_t*)crpyt_decrypt_data, sizeof(aes_128_gcm_pt_test0))) {
    printf("\ncrypt ct compare pass\n");
} else {
    printf("\ncrypt ct compare fail\n");
}
// 3.比对加密结果-tag
if(0 == memcmp((const uint8_t*)aes_128_gcm_tag_test0, (const uint8_t*)tag_sign
, sizeof(aes_128_gcm_tag_test0))) {
    printf("\nsign tag compare pass\n");
} else {
    printf("\nsign tag compare fail\n");
}
}
```

3.3.1.2.2. 解密和认证

```
void aes_128_gcm_demo_decrypt_verify(void)
{
    ErrStatus ret_error;
    // 1.解密和认证
    ret_error = aes_decrypt_verify((const cryptData_t*)(aes_128_gcm_ct_test0)
, sizeof(aes_128_gcm_pt_test0), CypherDataIsTheLast
, (plainData_t*)(crpyt_decrypt_data));
    if(ret_error != SUCCESS){
        printf("\naes_decrypt_verify return error\n");
        return;
    };
    // 2.比对解密结果-明文
    if(0 == memcmp((const uint8_t*)aes_128_gcm_pt_test0, (const uint8_t*)crpyt_decrypt_data
, sizeof(aes_128_gcm_ct_test0))) {
        printf("\ndecrypt pt compare pass\n");
    } else {
        printf("\ndecrypt pt compare fail\n");
    }
    // 3.比 tag 认证结果
    if(0 == memcmp((const uint8_t*)aes_128_gcm_tag_test0, (const uint8_t*)tag_verify
, sizeof(aes_128_gcm_tag_test0))) {
        printf("\nsign tag compare pass\n");
        /*状态标志位测试*/
        if(verify_status != AES_VERIFY_SUCCESS){
            printf("\nsign tag flag error\n");
        } else {
```

```
        printf("\nsign tag flag pass\n");
    }
} else {
    printf("\nsign tag compare fail\n");
    /*状态标志位*/
    if(verify_status != AES_VERIFY_FAIL){
        printf("\nsign tag flag error\n");
    } else {
        printf("\nsign tag flag pass\n");
    }
}
}
```

3.3.1.3. 分多次完成加密解密

过长的数据或者流式数据，可以分成多次完成加解密。

注意：中间数据（非最后一组数据），进行加解密时候需要数据长度需要=N*16bytes 的关系。

3.3.1.3.1. 加密和签名

```
void aes_128_gcm_demo_crypt_sign_mulit_times(void)
{
    ErrStatus ret_error ;
    //1 先完成一部分数据加密，16 字节的倍数
    ret_error = aes_crypt_sign((const plainData_t*)(aes_128_gcm_pt_test0), 16
    , CypherDatasNotTheLast, (cryptData_t*)(crpyt_decrypt_data));
    if(ret_error != SUCCESS){
        printf("\naes_crypt_sign return error\n");
        return;
    };
    //2 完成剩余数据的加密和签名
    ret_error = aes_crypt_sign((const plainData_t*)(&aes_128_gcm_pt_test0[16])
    , sizeof(aes_128_gcm_pt_test0) - 16, CypherDatisTheLast
    , (cryptData_t*)(&crpyt_decrypt_data[16]));
    if(ret_error != SUCCESS){
        printf("\naes_crypt_sign return error\n");
        return;
    };
    //3 比对加密结果-密文
    if(0 == memcmp((const uint8_t*)aes_128_gcm_ct_test0
    , (const uint8_t*)crpyt_decrypt_data, sizeof(aes_128_gcm_pt_test0))) {
        printf("\ncrypt ct compare pass\n");
    } else {
        printf("\ncrypt ct compare fail\n");
    }
    //4 比对加密结果-tag
```

```
if(0 == memcmp((const uint8_t*)aes_128_gcm_tag_test0
, (const uint8_t*)tag_sign, sizeof(aes_128_gcm_tag_test0))) {
    printf("\nsign tag compare pass\n");
} else {
    printf("\nsign tag compare fail\n");
}
}
```

3.3.1.3.2. 解密和认证

```
void aes_128_gcm_demo_crypt_sign_mulit_times(void)
{
    ErrStatus ret_error ;
    //1 先完成一部分数据加密，16 字节的倍数
    ret_error = aes_crypt_sign((const plainData_t*)(aes_128_gcm_pt_test0)
, 16, CypherDatasNotTheLast, (cryptData_t*)(crpyt_decrypt_data));
    if(ret_error != SUCCESS){
        printf("\naes_crypt_sign return error\n");
        return;
    };
    //2 完成剩余数据的加密和签名
    ret_error = aes_crypt_sign((const plainData_t*)&aes_128_gcm_pt_test0[16]
, sizeof(aes_128_gcm_pt_test0) - 16, CypherDatisIsTheLast
, (cryptData_t*)&crpyt_decrypt_data[16]));
    if(ret_error != SUCCESS){
        printf("\naes_crypt_sign return error\n");
        return;
    };
    //3 比对加密结果-密文
    if(0 == memcmp((const uint8_t*)aes_128_gcm_ct_test0
, (const uint8_t*)crpyt_decrypt_data, sizeof(aes_128_gcm_pt_test0))) {
        printf("\ncrypt ct compare pass\n");
    } else {
        printf("\ncrypt ct compare fail\n");
    }
    //4 比对加密结果-tag
    if(0 == memcmp((const uint8_t*)aes_128_gcm_tag_test0
, (const uint8_t*)tag_sign, sizeof(aes_128_gcm_tag_test0))) {
        printf("\nsign tag compare pass\n");
    } else {
        printf("\nsign tag compare fail\n");
    }
}
```

3.3.1.4. 测试函数

```
void aes_128_gcm_demo_test(void)
{
    printf("\n -----aes_128_gcm_demo_test begin ----- \n");
    /*加密场景 1 单次初始化, 单次加密*/
    aes_128_gcm_demo_init(AES_enCrypt);
    aes_128_gcm_demo_crypt_sign();

    /*一组数据, 分多次加密签名*/
    aes_128_gcm_demo_init(AES_enCrypt);
    aes_128_gcm_demo_crypt_sign_multitimes();

    /*加密场景 2 单次初始化, 多次加密*/
    aes_128_gcm_demo_init(AES_enCrypt);
    aes_128_gcm_demo_crypt_sign();
    /*只变化加密数据, 其他配置不变, 可以连续进行加密*/
    aes_128_gcm_demo_crypt_sign();

    /*解密场景 1 单次初始化, 单次解密*/
    aes_128_gcm_demo_init(AES_deCrypt);
    aes_128_gcm_demo_decrypt_verify();

    /*一次初始化化, 同一类型数据, 多帧多次解密*/
    aes_128_gcm_demo_init(AES_deCrypt);
    aes_128_gcm_demo_decrypt_verify();
    aes_128_gcm_demo_decrypt_verify();

    /*一组数据, 分多次解密认证*/
    aes_128_gcm_demo_init(AES_deCrypt);
    aes_128_gcm_demo_decrypt_verify_multitimes();

    printf("\n -----aes_128_gcm_demo_test finish ----- \n");
}
```

3.3.2. GMAC 函数 Demo

3.3.2.1. case 结构体和全局变量

```
typedef struct{
    key_t          *key;      //指针指向密钥
    iv_t           *iv;       //指针指向初始向量
    uint32_t        ivSize;    //初始向量长度。单位规定为 bytes
    /*指针指向 TAG, 加密时候用于结果比对,解密时候作为 TAG 数据输入认证比对和结果比对。*/
    tag_t          *tag;
```



```
uint32_t          tagSize;
AES_BITWIDE_MODE  bitWideMode; //AES_128/AES_192/AES_256
}SEA_AES_GMAC_ADD;
```

```
uint8_t          gmac_demo_tag_sign[16] ;           //签名信息
uint8_t          gmac_demo_tag_verify[16];          //解密签名认证信息
AES_VERIFY_STA   gmac_demo_verify_status;          //解密认证结果标志
```

3.3.2.2. case 信息

标准字符串输入数据(文本数据, 单纯参考使用)

"key": "1F19503EDC1C78EECBDA3906ECBC9C9F"

"pt": ""

"add": "F74A0CBE95B6B003059727283D57E2E556BB9BF12C96D4B7C8F99E509FFD81A6967511"

"iv": "A08559C3386F154AA7B5F43B96E8F5E24A51CB6F09EB911758EF"

标准字符串输出数据(文本数据)

"ct": ""

"tag": "8D636459677E544F2E78EF8FF5"

#ifdef AES_CRYPT_ITEMS_IN_BYTES

```
uint8_t aes_gmac_add_test0[] = {0xF7,0x4A,0x0C,0xBE,0x95,0xB6,0xB0,0x03,0x05,0x97,
0x27,0x28,0x3D,0x57,
0xE2,0xE5,0x56,0xBB,0x9B,0xF1,0x2C,0x96,0xD4,0xB7,0xC8,0xF9,0x9E,0x50,0x9F,0xFD,
0x81,0xA6,0x96,0x75,0x11};
```

```
uint8_t aes_gmac_iv_test0[] = {0xA0,0x85,0x59,0xC3,0x38,0x6F,0x15,0x4A,0xA7
,0xB5,0xF4,0x3B,0x96,0xE8,0xF5,0xE2,0x4A,0x51,0xCB,0x6F,0x09,0xEB,0x91,0x17,0x58,0xEF};
```

#else

```
__align(4) uint8_t aes_gmac_add_test0[] = {0xF7,0x4A,0x0C,0xBE,0x95,0xB6,0xB0,
0x03,0x05,0x97,0x27,0x28,0x3D
,0x57,0xE2,0xE5,0x56,0xBB,0x9B,0xF1,0x2C,0x96,0xD4,0xB7,0xC8,0xF9,0x9E,0x50,0x9F,
0xFD,0x81,0xA6,0x96,0x75,0x11};
```

```
__align(4) uint8_t aes_gmac_iv_test0[] = {0xA0,0x85,0x59,0xC3,0x38,0x6F,0x15,0x4A,0xA7
,0xB5,0xF4,0x3B,0x96,0xE8,0xF5,0xE2,0x4A,0x51,0xCB,0x6F,0x09,0xEB,0x91,0x17,0x58,0xEF};
```

#endif

```
uint8_t aes_gmac_key_test0[] = {0x1F,0x19,0x50,0x3E,0xDC,0x1C,0x78,0xEE,0xCB,0xDA,
0x39,0x06,0xEC,0xBC,0x9C,0x9F};
```

```
uint8_t aes_gmac_tag_test0[] = {0x8D,0x63,0x64,0x59,0x67,0x7E,0x54,0x4F,0x2E,0x78,
0xEF,0x8F,0xF5};
```

```
const SEA_AES_GMAC_ADD aes_gmac_demo_case_0={
    (uint32_t *)aes_gmac_key_test0 ,//uint32_t *key;
    (uint32_t *)aes_gmac_iv_test0   ,//uint32_t *iv;
    sizeof(aes_gmac_iv_test0)       ,//uint32_t ivSize;
    (tag_t *)aes_gmac_tag_test0 ,//uint32_t *tag;
    sizeof(aes_gmac_tag_test0)      ,//uint32_t tagSize;
```

```
AES_128                                     //AES_BITWIDE_MODE bitWideMode;
};
```

3.3.2.3. 测试函数

3.3.2.3.1. 初始化

```
ErrStatus aes_gmac_demo_init(AES_ENCRYPT_SEL encrypt_sel
, const SEA_AES_GMAC_ADD *p_testcase)
{
    ErrStatus ret_error = SUCCESS;
    aesCypherConfigStr aes_gcm_cfg ;

    //1 配置参数初始化
    aes_gcm_cfg.encrypt_sel      =  encrypt_sel;
    aes_gcm_cfg.bitWideMode      =  p_testcase->bitWideMode;
    aes_gcm_cfg.cryptMode        =  AES_GCM      ;
    aes_gcm_cfg.keyLocation      =  (key_t *)p_testcase->key;
    //aes_gcm_cfg.keySize        =  0; //not used
    aes_gcm_cfg.ivLocation       =  (iv_t *)p_testcase->iv;
    aes_gcm_cfg.ivSize           =  p_testcase->ivSize  ;

    aes_gcm_cfg.tagSignBuf       =  (uint32_t *)gmac_demo_tag_sign;
    aes_gcm_cfg.tagVerifyBuf     =  (uint32_t *)gmac_demo_tag_verify;
    aes_gcm_cfg.tagSize          =  p_testcase->tagSize ;
    aes_gcm_cfg.tagVerifySta     =  &gmac_demo_verify_status;
    aes_gcm_cfg.specialMode      =  AES_SPC_NONE;

    if(AES_deCrypt == encrypt_sel){
        memcpy(aes_gcm_cfg.tagSignBuf, p_testcase->tag, aes_gcm_cfg.tagSize);
    }
    //2 初始化
    if( SUCCESS != aes_crypt_init((const aesCypherConfigStr*) &aes_gcm_cfg)){
        ret_error = ERROR;
        printf("\naes_crypt_init - gmac return error\n");
    };
    return ret_error;
}
```

3.3.2.3.2. GMAC 签名

```
//分多次完成完整数据的签名
ErrStatus aes_gmac_demo_sign_mulit_times(uint8_t ghashData[]
, uint32_t size, const SEA_AES_GMAC_ADD *p_testcase)
{
    ErrStatus ret_error = SUCCESS;
```

```
//1 先完成一部分数据加密，16 字节的倍数
if( SUCCESS != aes_gmac((const plainData_t*)(ghashData)
, 16, CypherDatasNotTheLast)){
    printf("\naes_gmac sign block 1 return error\n");
    ret_error = ERROR;
};

//2 完成剩余数据的加密和签名
if( SUCCESS != aes_gmac((const plainData_t*)&ghashData[16])
, size - 16, CypherDatisTheLast)){
    printf("\naes_gmac sign block last return error\n");
    ret_error = ERROR;
};

//3 比对加密结果-tag
if(0 == memcmp(p_testcase->tag,
(const uint8_t*)gmac_demo_tag_sign, p_testcase->tagSize)){
    printf("\ngmac sign tag compare pass\n");
} else {
    printf("\ngmac sign tag compare fail\n");
}
return ret_error;
}
```

3.3.2.3.3. GMAC 认证

//分多次完成完整数据认证

```
ErrStatus aes_gmac_demo_verify_multi_times(uint8_t ghashData[], uint32_t size
, const SEA_AES_GMAC_ADD *p_testcase)
```

```
{
    ErrStatus ret_error = SUCCESS;
    //1 先完成一部分数据加密，16 字节的倍数
    if( SUCCESS != aes_gmac((const plainData_t*)(ghashData)
, 16, CypherDatisNotTheLast)){
        printf("\naes_gmac verify block 1 return error\n");
        ret_error = ERROR;
    };

    //2 完成剩余数据的加密和签名
    if( SUCCESS != aes_gmac((const plainData_t*)&ghashData[16])
, size - 16, CypherDatisTheLast)){
        printf("\naes_gmac verify block last return error\n");
        ret_error = ERROR;
    };

    //3 比对加密结果-tag
    if(0 == memcmp(p_testcase->tag, (const uint8_t*)gmac_demo_tag_verify
, p_testcase->tagSize)){
        printf("\ngmac verify tag compare pass\n");
    }
```

```
/*状态标志位*/
if(gmac_demo_verify_status != AES_VERIFY_SUCCESS){
    printf("\n gmac verify tag flag error\n");
    ret_error = ERROR;
}
}else{
    printf("\ngmac verify tag compare fail\n");
    ret_error = ERROR;
/*状态标志位*/
if(gmac_demo_verify_status != AES_VERIFY_FAIL){
    printf("\n gmac verify tag flag error\n");
}
}
return ret_error;
}
```

3.3.2.3.4. GMAC demo test

```
void aes_gmac_demo_test(void)
{
    WdtFeed();
    printf("\n -----aes_gmac_demo_test begin ----- \n");

    /*gmac encrypt initial*/
    aes_gmac_demo_init(AES_enCrypt, &aes_gmac_demo_case_0);
    //分多次完成完整数据的签名
    aes_gmac_demo_sign_mulit_times(aes_gmac_add_test0
    , sizeof(aes_gmac_add_test0), &aes_gmac_demo_case_0);

    /*gmac decrypt initial*/
    aes_gmac_demo_init(AES_deCrypt, &aes_gmac_demo_case_0);
    //分多次完成完整数据认证
    aes_gmac_demo_verify_multi_times(aes_gmac_add_test0
    , sizeof(aes_gmac_add_test0), &aes_gmac_demo_case_0);

    printf("\n -----aes_gmac_demo_test finish ----- \n");
}
```

3.3.3. 密钥封装和解封装 Demo

3.3.3.1. case 结构体

```
typedef struct {
    uint32_t *kek;
    uint32_t *key;
    uint32_t keylen;
```

```
uint32_t *ct;  
uint32_t ctlen;  
AES_BITWIDE_MODE bitWideMode;  
AES_CRYPT_MODE cryptMode;  
AES_KW_MODE keyWrapMode;  
} SEA_AES_KEYWRAP_TESTLIST;
```

3.3.3.2. 典型数据

```
/* RFC 3394 */  
const uint8_t kek1[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,  
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU};  
const uint8_t key1[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,  
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU};  
const uint8_t ct1[] = {0x1FU, 0xA6U, 0x8BU, 0x0AU, 0x81U, 0x12U, 0xB4U, 0x47U, 0xAEU, 0xF3U,  
0x4BU, 0xD8U, 0xFBU, 0x5AU, 0x7BU, 0x82U, 0x9DU, 0x3EU, 0x86U, 0x23U, 0x71U, 0xD2U, 0xCFU,  
0xE5U};  
  
const uint8_t kek2[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,  
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU, 0x10U, 0x11U, 0x12U, 0x13U, 0x14U, 0x15U, 0x16U, 0x17U};  
const uint8_t key2[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,  
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU};  
const uint8_t ct2[] = {0x96U, 0x77U, 0x8BU, 0x25U, 0xAEU, 0x6CU, 0xA4U, 0x35U, 0xF9U, 0x2BU,  
0x5BU, 0x97U, 0xC0U, 0x50U, 0xAEU, 0xD2U, 0x46U, 0x8AU, 0xB8U, 0xA1U, 0x7AU, 0xD8U, 0x4EU,  
0x5DU};  
  
const uint8_t kek3[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,  
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU, 0x10U, 0x11U, 0x12U, 0x13U, 0x14U, 0x15U, 0x16U, 0x17U,  
0x18U, 0x19U, 0x1AU, 0x1BU, 0x1CU, 0x1DU, 0x1EU, 0x1FU};  
const uint8_t key3[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,  
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU};  
const uint8_t ct3[] = {0x64U, 0xE8U, 0xC3U, 0xF9U, 0xCEU, 0x0FU, 0x5BU, 0xA2U, 0x63U, 0xE9U,  
0x77U, 0x79U, 0x05U, 0x81U, 0x8AU, 0x2AU, 0x93U, 0xC8U, 0x19U, 0x1EU, 0x7DU, 0x6EU, 0x8AU,  
0xE7U};  
  
const uint8_t kek4[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,  
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU, 0x10U, 0x11U, 0x12U, 0x13U, 0x14U, 0x15U, 0x16U, 0x17U};  
const uint8_t key4[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,  
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU, 0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U,  
0x07U};  
const uint8_t ct4[] = {0x03U, 0x1DU, 0x33U, 0x26U, 0x4EU, 0x15U, 0xD3U, 0x32U, 0x68U, 0xF2U,  
0x4EU, 0xC2U, 0x60U, 0x74U, 0x3EU, 0xDCU, 0xE1U, 0xC6U, 0xC7U, 0xDDU, 0xEEU, 0x72U, 0x5AU,  
0x93U, 0x6BU, 0xA8U, 0x14U, 0x91U, 0x5CU, 0x67U, 0x62U, 0xD2U};  
  
const uint8_t kek5[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,
```

```
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU, 0x10U, 0x11U, 0x12U, 0x13U, 0x14U, 0x15U, 0x16U, 0x17U,
0x18U, 0x19U, 0x1AU, 0x1BU, 0x1CU, 0x1DU, 0x1EU, 0x1FU};
```

```
const uint8_t key5[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU, 0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U,
0x07U};
```

```
const uint8_t ct5[] = {0xA8U, 0xF9U, 0xBCU, 0x16U, 0x12U, 0xC6U, 0x8BU, 0x3FU, 0xF6U, 0xE6U,
0xF4U, 0xFBU, 0xE3U, 0x0EU, 0x71U, 0xE4U, 0x76U, 0x9CU, 0x8BU, 0x80U, 0xA3U, 0x2CU, 0xB8U, 0x95U,
0x8CU, 0xD5U, 0xD1U, 0x7DU, 0x6BU, 0x25U, 0x4DU, 0xA1U};
```

```
const uint8_t kek6[] = {0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U, 0x07U, 0x08U, 0x09U,
0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU, 0x10U, 0x11U, 0x12U, 0x13U, 0x14U, 0x15U, 0x16U, 0x17U,
0x18U, 0x19U, 0x1AU, 0x1BU, 0x1CU, 0x1DU, 0x1EU, 0x1FU};
```

```
const uint8_t key6[] = {0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U, 0x88U, 0x99U,
0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU, 0x00U, 0x01U, 0x02U, 0x03U, 0x04U, 0x05U, 0x06U,
0x07U, 0x08U, 0x09U, 0x0AU, 0x0BU, 0x0CU, 0x0DU, 0x0EU, 0x0FU};
```

```
const uint8_t ct6[] = {0x28U, 0xC9U, 0xF4U, 0x04U, 0xC4U, 0xB8U, 0x10U, 0xF4U, 0xCBU, 0xCCU,
0xB3U, 0x5CU, 0xFBU, 0x87U, 0xF8U, 0x26U, 0x3FU, 0x57U, 0x86U, 0xE2U, 0xD8U, 0x0EU, 0xD3U, 0x26U,
0xCBU, 0xC7U, 0xF0U, 0xE7U, 0x1AU, 0x99U, 0xF4U, 0x3BU, 0xFBU, 0x98U, 0x8BU, 0x9BU, 0x7AU,
0x02U, 0xDDU, 0x21U};
```

```
/* RFC 5649 */
```

```
const uint8_t kek7[] = {0x58, 0x40, 0xDF, 0x6E, 0x29, 0xB0, 0x2A, 0XF1, 0xAB, 0x49, 0x3B, 0x70, 0x5B,
0xF1, 0x6E, 0XA1, 0xAE, 0x83, 0x38, 0xF4, 0xDC, 0xC1, 0x76, 0XA8};
```

```
const uint8_t key7[] = {0xC3, 0x7B, 0x7E, 0x64, 0x92, 0x58, 0x43, 0x40, 0xBE, 0xD1, 0x22, 0x07, 0x80,
0x89, 0x41, 0x15, 0x50, 0x68, 0xF7, 0x38};
```

```
const uint8_t ct7[] = {0x13, 0x8B, 0xDE, 0xAA, 0x9B, 0x8F, 0xA7, 0xFC, 0x61, 0xF9, 0x77, 0x42, 0xE7,
0x22, 0x48, 0xEE, 0x5A, 0xE6, 0xAE, 0x53, 0x60, 0xD1, 0xAE, 0x6A, 0x5F, 0x54, 0xF3, 0x73, 0xFA, 0x54,
0x3B, 0x6A};
```

```
const uint8_t kek8[] = {0x58, 0x40, 0xDF, 0x6E, 0x29, 0xB0, 0x2A, 0XF1, 0xAB, 0x49, 0x3B, 0x70, 0x5B,
0xF1, 0x6E, 0XA1, 0xAE, 0x83, 0x38, 0xF4, 0xDC, 0xC1, 0x76, 0XA8};
```

```
const uint8_t key8[] = {0x46, 0x6F, 0x72, 0x50, 0x61, 0x73, 0x69};
```

```
const uint8_t ct8[] = {0xAF, 0xBE, 0xB0, 0xF0, 0x7D, 0xFB, 0xF5, 0x41, 0x92, 0x00, 0xF2, 0xCC, 0xB5,
0x0B, 0xB2, 0x4F};
```

3.3.3.3. 测试函数

```
ErrStatus aes_keywrap_unit(SEA_AES_KEYWRAP_TESTLIST *testlist) {
    uint8_t cipher[48];
    uint8_t plain[48];
    uint32_t cipherlen, plainlen;

    sea_memset_word((uint32_t *)cipher, 0U, 12U);
    sea_memset_word((uint32_t *)plain, 0U, 12U);
```

```
    aesCypherConfigStr aes_gcm_cfg;
    aes_gcm_cfg.encrypt_sel = AES_enCrypt;
    aes_gcm_cfg.bitWideMode = testlist->bitWideMode;
    aes_gcm_cfg.cryptMode = testlist->cryptMode;
    aes_gcm_cfg.keyLocation = (key_t *)testlist->kek;
    aes_gcm_cfg.keyWrapMode = testlist->keyWrapMode;

    if (SUCCESS != LL_AES_crypt_init((const aesCypherConfigStr *)&aes_gcm_cfg)) {
        SEA_MSG("\n[log] LL_AES_crypt_init1 ERROR!!!");
        return ERROR;
    }
    if (SUCCESS != LL_AES_key_wrap((const plainData_t *)testlist->key, testlist->keylen, (cryptData_t
*)cipher, &cipherlen)) {
        SEA_MSG("\n[log] LL_AES_key_wrap ERROR!!!");
        return ERROR;
    }

    if ((testlist->ctlen != cipherlen) || (0U != memcmp((uint8_t *)cipher, (uint8_t *)testlist->ct, testlist->ctlen))) {
        SEA_MSG("\n[log] key wrap ERROR!!!");

        SEA_MSG("\n ctlen=%d, cipherlen=%d", testlist->ctlen, cipherlen);
        SEA_MSG("\n cipher:\t");
        for (int i = 0; i < cipherlen; i++) {
            SEA_MSG("%02X ", cipher[i]);
        }
        SEA_MSG("\n      ct:\t");
        uint8_t *tt = (uint8_t *)testlist->ct;
        for (int i = 0; i < cipherlen; i++) {
            SEA_MSG("%02X ", tt[i]);
        }
        return ERROR;
    }

    aes_gcm_cfg.encrypt_sel = AES_deCrypt;
    aes_gcm_cfg.bitWideMode = testlist->bitWideMode;
    aes_gcm_cfg.cryptMode = testlist->cryptMode;
    aes_gcm_cfg.keyLocation = (key_t *)testlist->kek;
    aes_gcm_cfg.keyWrapMode = testlist->keyWrapMode;

    if (SUCCESS != LL_AES_crypt_init((const aesCypherConfigStr *)&aes_gcm_cfg)) {
        SEA_MSG("\n[log] LL_AES_crypt_init ERROR!!!");
        return ERROR;
    }
}
```

```
if (SUCCESS != LL_AES_key_unwrap((const cryptData_t *)cipher, cipherlen, (plainData_t *)plain,
&plainlen)) {
    SEA_MSG("\n[log] LL_AES_key_unwrap ERROR!!!");
    return ERROR;
}

if ((testlist->keylen != plainlen) || (0U != memcmp((uint8_t *)plain, (uint8_t *)testlist->key, testlist->keylen)))
{
    SEA_MSG("\n[log] key unwrap ERROR!!!");

    SEA_MSG("\n ptlen=%d, plainlen=%d", testlist->keylen, plainlen);
    SEA_MSG("\n plain:\t");
    for (int i = 0; i < plainlen; i++) {
        SEA_MSG("%02X ", plain[i]);
    }
    SEA_MSG("\n      pt:\t");
    uint8_t *tt1 = (uint8_t *)testlist->key;
    for (int i = 0; i < plainlen; i++) {
        SEA_MSG("%02X ", tt1[i]);
    }
    return ERROR;
}

return SUCCESS;
}
```

3.3.4. AAD 多次传入 demo

3.3.4.1. case 结构体

```
typedef struct {
    uint32_t *pt;           /* plaintext */
    uint32_t ptlen;        /* ptSize in bytes */
    uint32_t *key;         /* key */
    uint32_t keylen;       /* keySize in bytes */
    uint32_t *iv;          /* IV(Initialization Vector) */
    uint32_t ivlen;        /* ivSize in bytes, AES_CTR_ivSize=16 */
    uint32_t *aad;         /* AAD(Additional Authenticated Data) */
    uint32_t aadlen;       /* addSize in bytes */
    uint32_t *ct;          /* ciphertext */
    uint32_t *tag;         /* signature */
    uint32_t taglen;       /* tagSize in bytes */
    AES_BITWISE_MODE bitwidemode; /* bitWideMode */
    AES_CRYPT_MODE cryptmode;    /* cryptMode */
} AES_AAD_TESTLIST;
```


3.3.4.2. 典型数据

```
const uint8_t aes_aad_test_pt1[] = {"87D684E7C0FC3FBF1B0B3ED69076AEC3"};
const uint8_t aes_aad_test_ky1[] = {"F0BAD348BCAD3CE36A1F29E43FAC5FC328AEB48CED0B2B60"};
const          uint8_t          aes_aad_test_ad1[]          =
{"8F1B9187CD629473C584D8EFF104E99AEE0EC01A12B71C6A0CD09D85DCEE1247BC9CBC2D87455C39
03108EBA135DD059596B0BAEDC8CB94C4818CC2A9B9784533EEB4CC25E4306"

"F9BD9DA7CB036583D9FB391D61425032095E1A708D3F19D7AADC911F9636CC9277CD4EFF0F951767"};
const uint8_t aes_aad_test_iv1[] = {"B9A50B44724261E55E81838E"};
const uint8_t aes_aad_test_ct1[] = {"4F6A973D7F5A7DB3FBC8569AF8421BAB"};
const uint8_t aes_aad_test_tg1[] = {"97B55DB09E748B1263860E007543C18D"};

const uint8_t aes_aad_test_pt2[] = {"B7C8B39E4067140C5EB4046306EB095D"};
const uint8_t aes_aad_test_ky2[] = {"2ACCF73AA977E68E9555B730B36E402AFF7AF802D6DFBF9F"};
const          uint8_t          aes_aad_test_ad2[]          =
{"CF02D75E8EA735FAE8617D4230627952C93C2655A703C8439B6F915CB09FA54F41761F498D2D04296069
DEA9AB07FF3330C785DEEB02A0EE4C658C92D09564C70A29492F95A91A"

"4E1A125848F215A8270262F21466B0798AA5B4F3A87C1E3BB8B1D1BE542E019522DCEC8F658D46A3"};
const uint8_t aes_aad_test_iv2[] = {"400E08B10C4E1698C97E7831"};
const uint8_t aes_aad_test_ct2[] = {"53B7E7B7990EF3229A3680B09A4FA9B1"};
const uint8_t aes_aad_test_tg2[] = {"36592790E23CEA477BAD88D9E6F69EFD"};

const uint8_t aes_aad_test_pt3[] = {"D8061D4BCCEE1DE52A255D7C3550965D"};
const uint8_t aes_aad_test_ky3[] = {"B28C50B033CADB95C7F2BFDA5E6B51FD6C069506550F29B5"};
const          uint8_t          aes_aad_test_ad3[]          =
{"3C964D8048AAC566256F196AF6ECEC6C75D72D0F15BCAE29FA9C218C50D5AA77EF3B9B3C4B5B2E5
164D492A86279B3D474AC56CCD4FC4DD7D398DC77FB2A3AB8187CE3A9C06018"

"CCA4010B629382C2D6131DB78B8DC3F3D3E45C9D6427D3537037262A254FC6F12B38E48FC8CF3233"};
const uint8_t aes_aad_test_iv3[] = {"70FD034A4E9A72DE43A13370"};
const uint8_t aes_aad_test_ct3[] = {"4420263AC4088824BC7058B3C12E6FCE"};
const uint8_t aes_aad_test_tg3[] = {"1FB35507291DE55229A7751CBC2E7E37"};

const uint8_t aes_aad_test_pt4[] = {"1C3A2D1E246131CF378DD353B9496C23"};
const uint8_t aes_aad_test_ky4[] = {"2D7BE3EDEA0352C3DDD03E163AAA5E7962B2941A4FCD7294"};
const          uint8_t          aes_aad_test_ad4[]          =
{"7C6F741D45403DC3DF55904F976639A5C10BD1196F0F18885D2393C06B65D6D55D339A710D281EB5AB
125C68867ECC9E4CFE4525A306DE17E0824563F6E9D63CBD16CD630585A3"

"063074204D26F80AA2868AA941A194EC2B979FF7B12E4F29CC60836091215328F540E7BA8F3175B6"};
const uint8_t aes_aad_test_iv4[] = {"D73374B320B537C01EA7EFB7"};
```

```
const uint8_t aes_aad_test_ct4[] = {"48F44629A27EF037B3AD73B19BB85B6D"};
const uint8_t aes_aad_test_tg4[] = {"866847C10A3B4900641EE775C240961B"};

const uint8_t aes_aad_test_pt5[] = {"784ECB4D41E44B88521055BDF73747A9"};
const uint8_t aes_aad_test_ky5[] = {"CA401E3B90BF669EEFA91E61D67EA807655A32BA0A77A8BE"};
const
uint8_t
aes_aad_test_ad5[]
=
{"07F4E69D2C8E53FC55E75A97583E02E13DFAF95629C0E1CE438BCE16E0DCCC8A204FB93B71D31AC38
2C5748F443C67B5284C2BA94E0DB2C13058704652D5C2C9769ED4C0F32B2E"

"69A34763DC811226B3D120ECA2A64329F410AD67A03DBE0BE6A2F5A16AF039D615EFD1EC7702C26E"}
;
const uint8_t aes_aad_test_iv5[] = {"97F8AD045EBC64BF1708C5FA"};
const uint8_t aes_aad_test_ct5[] = {"D8F4AFF291001AD98969B90648486DB4"};
const uint8_t aes_aad_test_tg5[] = {"812F53C8291582AF6668012F3BA62419"};

const uint8_t aes_aad_test_pt6[] = {"5531BD8F790A91747C4BCFF68BC7A6A2"};
const uint8_t aes_aad_test_ky6[] = {"6EE1B9B604E3CBE4AC855B9177B8A96B4045E645F494F73D"};
const
uint8_t
aes_aad_test_ad6[]
=
{"64B81167397F4718D18123E67FEABE2DAA26D411881DFFDD44C2770593258E06EF86CF80EB708AB328
080CD33FCF4D6CD66B837C528A043438FCD7C3337021DE5562B5CD569599"

"2EA3269B8C5237DA1B86AA75B503E469A4532615849F2DDAC4955E0B19EAD3174C208761BC2F575A"};
const uint8_t aes_aad_test_iv6[] = {"4E8176F10D34B63C617CA0A1"};
const uint8_t aes_aad_test_ct6[] = {"85A8170C38780CACF0549776B2EA35F6"};
const uint8_t aes_aad_test_tg6[] = {"B6DCE3F6255C8C3FFB7B6F3AEA274CC6"};

const uint8_t aes_aad_test_pt7[] = {"D42B92DA747A0B9E3D07720C5443C335"};
const uint8_t aes_aad_test_ky7[] = {"24B2A0ADB8AF5BE8692B3A46817C06665B8899AC0E5960E2"};
const
uint8_t
aes_aad_test_ad7[]
=
{"CC23663FC18686BE1A48D1C8257D0E1D611C7BE1DC94350CE1A2ED45D6BC0CD234E304008F5FF5CF
9AC3AC8560772DB19E9564609702EBD024F59FB6562182C6CE313F522FADB2"

"C67D13E4C7645902805843675883DA04743634AE422672E27E8B11DF63F786F505EDE73B9B772104"};
const uint8_t aes_aad_test_iv7[] = {"D238C528399F0DD8AD9537D8"};
const uint8_t aes_aad_test_ct7[] = {"9D7D8C62F521627BED0F58B97E532E20"};
const uint8_t aes_aad_test_tg7[] = {"F25C5144EF022AA5971CA502F077EB46"};

const uint8_t aes_aad_test_pt8[] = {"FE0E7B5685FA95CE82CFE95B0A96AFCA"};
const uint8_t aes_aad_test_ky8[] = {"66D572A29D9558AE37D4F6280D941AFD5173C61D3DD25024"};
const
uint8_t
aes_aad_test_ad8[]
=
{"7D6FBF9D2D15784D4A844962D8019CD38D969326801DAA2E9025DB77CC5F2C1EA80859E84BFF0D545
960CDBF9D2B600FF793CECBFD7EF6D7570B9D3B69D2AD1743E78D39119366"

"5C6291A476F4013A2E61454979776F0636416B7BB15C70E5FED6C2C1D759C155410C453B04B43743"};
```

```
const uint8_t aes_aad_test_iv8[] = {"D3BCFAAC4FD084DA04BFE7FB"};
const uint8_t aes_aad_test_ct8[] = {"9D87BDD5694FF8FFEEADF17F24FBBE24"};
const uint8_t aes_aad_test_tg8[] = {"0729EB7449FA7B78D0F9627FAF7129A7"};

const uint8_t aes_aad_test_pt9[] = {"7272B8D9EC9065BA5A31C0991DA39CA8"};
const uint8_t aes_aad_test_ky9[] = {"545AD91F7152813353B9B32BA3B03DCD12BEAF12A4CEE1B1"};
const
uint8_t
aes_aad_test_ad9[]
=
{"C885B600858A285A176BA0BE64E7F86858D6356BFADB18CE4FBB1BC649DF2E02308B36874FC46D0D6
78A6386321AE05C062036BC238633FE0D06C4F3978FE57AF1D2D94444FE89"

"8E9358F12EDA25D413AEB3552E5A20D509D07A86D852016658B93DAB072E8E0F5C4F90DDF98F0479"};
const uint8_t aes_aad_test_iv9[] = {"93DE17F4FDE79EB89DDA08D3"};
const uint8_t aes_aad_test_ct9[] = {"52EFA92A49A5B4BD61433DB8D3360A7A"};
const uint8_t aes_aad_test_tg9[] = {"03EB4BAA21D0B9149ACC9FE71035E5B5"};

const uint8_t aes_aad_test_pt10[] = {"42A1880A426F5F063246ABF0C603140B"};
const uint8_t aes_aad_test_ky10[] = {"F56F32E08C921FAA386A1326CA28C2D0AFF33AF9E7DC5953"};
const
uint8_t
aes_aad_test_ad10[]
=
{"A2A84774908967B5C08A490294620515F4F6C20DFA23DB9B2E44B50A66FE585BC14374A9E3D5A8347F
B2FCC72C93FE0F3C279775FF5DEC16512B4A38BEE668BD5275969372413"

"C8C6CC16AE63A2DC7797D73F1CFBBABD157D7E7EA206214F6EA9836F5F30C7C832DF65320D0B4C3F
E"};
const uint8_t aes_aad_test_iv10[] = {"3C4CAD4DB4D101F44869A20F"};
const uint8_t aes_aad_test_ct10[] = {"8F9173087BEAFF993E2A185587CB4BF5"};
const uint8_t aes_aad_test_tg10[] = {"6BAC1B16FAE57E157BC2C411A45CB610"};

const uint8_t aes_aad_test_pt11[] = {"AD93FE3CA016FA497DCB345BCDF6E15E"};
const uint8_t aes_aad_test_ky11[] = {"DB2FAE7FD6C5042756ECDA242AB648215F36DF256A232EC3"};
const
uint8_t
aes_aad_test_ad11[]
=
{"F869C36E2CEAE93055A2A13EA780E67A059916485349561B90F2C0A69FC80A322399E9E2124590F34C5
4956C7E15F0DE3687E3912EA1D1E24BD5B3BDDE3584EB0F1F54975E1C0"

"2E82B45EA242BA2F043B09EDC24B31951783148FE142DBC393778C00E6E59A07D12AC96AA7F19032A"}
;
const uint8_t aes_aad_test_iv11[] = {"4AEAE17A176CFDAC1D8DAD38"};
const uint8_t aes_aad_test_ct11[] = {"35ED8DF23E8F9CB5D32C73E54D350402"};
const uint8_t aes_aad_test_tg11[] = {"29FA94FAAB8A6EE0850812CC783CE439"};

const uint8_t aes_aad_test_pt12[] = {"5DD46FCAB6E5D574D69B1B42BBCD7D97"};
const uint8_t aes_aad_test_ky12[] = {"95F8B1A7851B8ACE0D1F6F44C7025F48574886355D3010C7"};
const
uint8_t
aes_aad_test_ad12[]
=
{"E4A40F70A7D81BD3C0F6807268F1929D125E68E8AD6DB48AA22C6AA4422EF45CC9B91D04D884FE49
A1BB685892FA008B81A11EF1A1B57F02A9E7F5A621BD8E284E126C687D143"}
```

```
"E31500A9E2C6F5D7198D28C61D157436F36229B1816B64E5440E8D8B60492D318E372753511ECA19D";
const uint8_t aes_aad_test_iv12[] = {"C2FEACF575FD76000416FF3B"};
const uint8_t aes_aad_test_ct12[] = {"8ADED0AFE90AC262D565709824F6C446"};
const uint8_t aes_aad_test_tg12[] = {"6744B00EAF2D4EE89DD4800648F67B48"};

const uint8_t aes_aad_test_pt13[] = {"1D1AF447AACF93D1032AF992D5646FE3"};
const uint8_t aes_aad_test_ky13[] = {"3AE877DD81006E9001B1366D3DF8CDBBE98C104479987DF6"};
const
    uint8_t
    aes_aad_test_ad13[]
    =
    {"5147222ECEE0CA46936E9D643899E54DCE770AD545B7A72E1711BB1974B1CCE8FDF4237222DCFAC4A
    24FA180287312A03D0F942723618575B9440BC9A56D9634EFC6D84B39235"

"D629E2E36CA2D617222DA60860AE6BBAED63F5D957F3F99F3D91ACEE19B96ED00C41A7AE3D237C5D
3"};
const uint8_t aes_aad_test_iv13[] = {"4FC44B629846B003D1616E39"};
const uint8_t aes_aad_test_ct13[] = {"56AE36FD60F7ADC8D3E827D0B685F2C7"};
const uint8_t aes_aad_test_tg13[] = {"0B6992D021E83FBF2426DFD8FCC685DF"};

const uint8_t aes_aad_test_pt14[] = {"FE68CDCBA2C3065450EBE92568BC2B3A"};
const
    uint8_t
    aes_aad_test_ky14[]
    =
    {"9365F2ABB4E6DA7429BD2D7E2271195863CA00E0CD4A8C3430AB2F534E7CCDA8"};
const
    uint8_t
    aes_aad_test_ad14[]
    =
    {"5307DF9CAE24F73A5ACED13BF829DF3932B3C565E83E4F0FDD8E30693AFF02245018A2274B3D75C7A
    D225AD2EA80C6857DB8B4095753A4F62F2AD924EBA45DB5A0BBBCBFAF5DF"

"2E6BC1DB98C357E3AB6B21B0D98B5A2DF0CD752853D2332E466AE8F50253679A18B5069AB375F7B48
B468FD56CF0A0C45F1E0F23851E9424F68"};
const
    uint8_t
    aes_aad_test_iv14[]
    =
    {"5418077D33A2B308EB18A4FA94C0FF0FA46E7F739A387FB60D83C7EA828CA3F46340F75F8AEC22E771
    F00789C25375A8DDBADE8C37B3A84B77926A568A24F9F2A1664D0968DDC"

"982FBB7274284F64878DA520F7B7F455BB1495F3C546F1D9F98338B800BCC4FFBED365ED2"};
const uint8_t aes_aad_test_ct14[] = {"4BC26A8CC11E131E5F40D46F87AE4C42"};
const uint8_t aes_aad_test_tg14[] = {"399B89E3144FF9EE"};

const uint8_t aes_aad_test_pt15[] = {"EBB0E55E9FF2DBB9180B4DAC5A70DB40"};
const
    uint8_t
    aes_aad_test_ky15[]
    =
    {"2471D0DDCA33099BAEA25C0DA692F5192B56B694BD33F275EBEB79670E9D3D43"};
const
    uint8_t
    aes_aad_test_ad15[]
    =
    {"6F99E9EF1B87635470EA7B1E5741C0AA5E39491249946FE855333F41745BECDDA13F9DF8ECAD01CE4
    F14FC64120EE8AA755F178D0EDDA40D91D59C1C56B3696C4F8D2D40EB350"

"A76C757F0EDA200BDF15096F4DEC83B6BCF80892DF623C9263F029E65D00039406A3DDFC979AC2496B
FCBBFE67DF36C0BB55E2D792CA0485590"};
```

```
const                uint8_t                aes_aad_test_iv15[]                =
{"D19C1239C089B13613E84AD3F0CAFCFAB6CE3E7859F8DB14D7255AB170545288B2BA44A81E4D8D0A
384075CE80E2DAB9DFEA80F2D0084A154530864D94E97789B7CBB5CB83DCE"

"436C00F81A40E20455D73EE9BE7DD09536885C78503834E19B9E484EFC63F4DA87AD15DFC"};
const uint8_t aes_aad_test_ct15[] = {"A7404DE432C03E6F66FFA1ECC00757D4"};
const uint8_t aes_aad_test_tg15[] = {"19663C202B80E5D3"};

const uint8_t aes_aad_test_pt16[] = {""};
const uint8_t aes_aad_test_ky16[] = {"A0D83382D913895F32AFBBBFB94CA91D"};
const                uint8_t                aes_aad_test_ad16[]                =
{"F4594739A94AEA9E3BBA8FEDBE04DBBE5196FABF882AA0A6A16AC8E6EFC72951E229AB7FEB4830C
99EC1F7F52BD2FA47CE5FE38F"};
const uint8_t aes_aad_test_iv16[] = {"5D9418"};
const uint8_t aes_aad_test_ct16[] = {""};
const uint8_t aes_aad_test_tg16[] = {"52FE67F75AE6BD66C73B720AC1680B"};

const uint8_t aes_aad_test_pt17[] = {""};
const uint8_t aes_aad_test_ky17[] = {"1F19503EDC1C78EECBDA3906ECBC9C9F"};
const                uint8_t                aes_aad_test_ad17[]                =
{"F74A0CBE95B6B003059727283D57E2E556BB9BF12C96D4B7C8F99E509FFD81A6967511"};
const uint8_t aes_aad_test_iv17[] = {"A08559C3386F154AA7B5F43B96E8F5E24A51CB6F09EB911758EF"};
const uint8_t aes_aad_test_ct17[] = {""};
const uint8_t aes_aad_test_tg17[] = {"8D636459677E544F2E78EF8FF5"};

const uint8_t aes_aad_test_pt18[] = {""};
const uint8_t aes_aad_test_ky18[] = {"8344816E93546368F6BF3ACF4FE16C77"};
const                uint8_t                aes_aad_test_ad18[]                =
{"B4B82548569E4636BA14BBFC13BB7184CD77FDED1B9699BE91F38BB39B8359FFF3BAB853AC95184A
313C1A73BF610E4F508AF559AA9E7F6C0CD1DE6EE97873D5887A5E210BC20"

"EC2D0A78C9B02A5906F62752B0F58E17A52DCC315BB58E9F2BDD81D194FD170C47C994D3B24F05B2F
228D3658DB354CF2A2CA2F92D03EA8100C"};
const uint8_t aes_aad_test_iv18[] = {"CC7DF732C71DA6BB51615B4B"};
const uint8_t aes_aad_test_ct18[] = {""};
const uint8_t aes_aad_test_tg18[] = {"79EBE4D46CBD21DC4EEDE72922D8"};

const uint8_t aes_aad_test_pt19[] = {""};
const uint8_t aes_aad_test_ky19[] = {"17979378F909FB32F6D6D8CF1DA347F6EE1D569E0A285515"};
const                uint8_t                aes_aad_test_ad19[]                =
{"8C62CB509144E7B2B457DA6D8C5036A6B43F3CF1FC8BD896F60AD0E939B84612A0CA24CE2C9139D1
0AB5B95D6BBFEAC4FBF55B1566B44AB131F7E2CA16F2686DB4452D7614319"

"B0F55A3A5463C0A8DD9D5E6DCA225FB58CD413900C8A18E74B0FCF567F998BFABD7978111BE828CE
```

```
A31C084598BA365631C3FC536C009339067"};
const uint8_t aes_aad_test_iv19[] = {"6127D3273C4E5D2F727F0789"};
const uint8_t aes_aad_test_ct19[] = {""};
const uint8_t aes_aad_test_tg19[] = {"D7ACFA6889C61B4C"};

const uint8_t aes_aad_test_pt20[] = {""};
const          uint8_t          aes_aad_test_ky20[]          =
{"81E975A14A38B04EB4E1D2C9FF647886F3D050CB7D1A6FA49BB5147452D71376"};
const          uint8_t          aes_aad_test_ad20[]          =
{"0989FBF4832AD28D8332D5EAAF2D938E78F9A240178BFFBA8D4C74DA7D3C98852EFA6BDCC3042331
A3C1BD4F138BF11CA422D6BA"};
const uint8_t aes_aad_test_iv20[] = {"3F92B17465BEF9A75C764E40"};
const uint8_t aes_aad_test_ct20[] = {""};
const uint8_t aes_aad_test_tg20[] = {"F3A82B6F0F154AB88EDDC4AE649B09"};
```

3.3.4.3. 测试函数

```
static ErrStatus aes_gcm_testunit(AES_AAD_TESTLIST *testlist, AES_ENCRYPT_SEL encrypt_sel, uint8_t
aad_mode) {
    ErrStatus tmp_err = SUCCESS;
#ifdef AES_CRYPT_ITEMS_IN_BYTES
    add_t aad[512];
#else
    add_t aad[128];
#endif

#ifdef AES_PROCESS_LATCH_RECOVER_SUPPORT
    aesProcessParameters tmp_latch_buf;
#endif

    uint32_t pt[32], ct[32], key[8], iv[64], tagSign[8], tagVer[8], output[32];
    uint32_t i, total_len, loops, calcSize, tmp_addr;
    AES_AAD_TESTLIST *pCase = testlist;
    aesCypherConfigStr testcfg;
    AES_VERIFY_STA verStat;

    sea_memset_word(pt, 0U, 32U);
    sea_memset_word(ct, 0U, 32U);
    sea_memset_word(output, 0U, 32U);
    sea_memset_word(key, 0U, 8U);
    sea_memset_word(iv, 0U, 64U);
    sea_memset_word(tagSign, 0U, 8U);
    sea_memset_word(tagVer, 0U, 8U);
    sea_memset_word((uint32_t *)aad, 0U, 128U);
```



```
string2hex((const uint8_t *)pCase->key, (uint8_t *)key, pCase->keylen);
string2hex((const uint8_t *)pCase->iv, (uint8_t *)iv, pCase->ivlen);
string2hex((const uint8_t *)pCase->aad, (uint8_t *)aad, pCase->aadlen);

testcfg.encrypt_sel = encrypt_sel;
testcfg.bitWideMode = pCase->bitwidemode;
testcfg.cryptMode = pCase->cryptmode;
testcfg.keyLocation = (key_t *)&key[0];
testcfg.ivLocation = (iv_t *)&iv[0];
testcfg.ivSize = pCase->ivlen;
if (aad_mode == 1) {
    testcfg.addLocation = (add_t *)&aad[0];
    testcfg.addSize = pCase->aadlen;
} else {
    testcfg.addLocation = (add_t *)0U;
    testcfg.addSize = 0U;
}
testcfg.tagSignBuf = (tag_t *)&tagSign[0];
testcfg.tagVerifyBuf = (tag_t *)&tagVer[0];
testcfg.tagSize = pCase->taglen;
testcfg.tagVerifySta = &verStat;
testcfg.specialMode = AES_SPC_NONE;

string2hex((const uint8_t *)pCase->pt, (uint8_t *)pt, pCase->ptlen);
string2hex((const uint8_t *)pCase->ct, (uint8_t *)ct, pCase->ptlen);
if (testcfg.encrypt_sel == AES_deCrypt) {
    string2hex((const uint8_t *)pCase->tag, (uint8_t *)tagSign, testcfg.tagSize);
} else {
    string2hex((const uint8_t *)pCase->tag, (uint8_t *)tagVer, testcfg.tagSize);
}

tmp_err = LL_AES_crypt_init((const aesCypherConfigStr *)&testcfg);
if (SUCCESS == tmp_err) {
    if (aad_mode == 0) {
        total_len = pCase->aadlen;
        loops = ((0 != (total_len % 16U)) ? ((total_len >> 4U) + 1U) : (total_len >> 4U));
        for (i = 0; i < loops; i++) {
            WdtFeed();
#endif AES_PROCESS_LATCH_RECOVER_SUPPORT
            if ((i == 0) || (i == 3)) {
                SEA_MSG("\n aes_latch recover begin\n");
                aes_process_status_latch(&tmp_latch_buf);
                aes_128_gcm_demo_init(AES_enCrypt);
                aes_128_gcm_demo_crypt_sign();
```

```
        aes_process_status_recover(&tmp_latch_buf);
        SEA_MSG("\n aes_latch recover finish\n");
    } else if (i == 1) {
        SEA_MSG("\n aes_latch recover begin\n");
        aes_process_status_latch(&tmp_latch_buf);
        aes_128_gcm_demo_init(AES_deCrypt);
        aes_128_gcm_demo_decrypt_verify();
        aes_process_status_recover(&tmp_latch_buf);
        SEA_MSG("\n aes_latch recover finish\n");
    }
#endif

    calcSize = ((total_len > 16U) ? 16U : total_len);
    total_len -= calcSize;

    tmp_addr = ((uint32_t)aad + (i * 16U));
    tmp_err = LL_AES_update_aad((const add_t *)tmp_addr, calcSize);
    if (SUCCESS != tmp_err) {
        SEA_MSG("\n[log]++update aad Fail!!!");
    }
}
}
} else {
    SEA_MSG("\n[log]++init Fail!!!\n");
}
if (SUCCESS == tmp_err) {
    if (AES_enCrypt == encrypt_sel) {
        tmp_err = LL_AES_crypt_sign((const plainData_t *)pt, pCase->ptlen, CypherDataIsTheLast,
(cryptData_t *)output);
    } else {
        tmp_err = LL_AES_decrypt_verify((const cryptData_t *)ct, pCase->ptlen, CypherDataIsTheLast,
(plainData_t *)output);
    }
}
(void)LL_AES_crypt_deinit();

if (SUCCESS == tmp_err) {
    if (AES_enCrypt == encrypt_sel) {
        if (0U != memcmp((const uint8_t *)output, (const uint8_t *)ct, pCase->ptlen)) {
            tmp_err = ERROR;
            SEA_MSG("\n[log]++output\tct");
            for (i = 0; i < (pCase->ptlen >> 2U); i++) {
                SEA_MSG("\n%08X %08X", output[i], ct[i]);
            }
        }
    }
}
```



```
    if (0U != memcmp((const uint8_t *)tagSign, (const uint8_t *)tagVer, pCase->taglen)) {
        tmp_err = ERROR;
        SEA_MSG("\n[log]++tagSign\ttagVer");
        for (i = 0; i < (pCase->taglen >> 2U); i++) {
            SEA_MSG("\n%08X %08X", tagSign[i], tagVer[i]);
        }
    }
} else {
    if (0U != memcmp((const uint8_t *)output, (const uint8_t *)pt, pCase->ptlen)) {
        tmp_err = ERROR;
        SEA_MSG("\n[log]++output\tct");
        for (i = 0; i < (pCase->ptlen >> 2U); i++) {
            SEA_MSG("\n%08X %08X", output[i], pt[i]);
        }
    }
}

    if (0U != memcmp((const uint8_t *)tagVer, (const uint8_t *)tagSign, pCase->taglen)) {
        tmp_err = ERROR;
        SEA_MSG("\n[log]++tagSign\ttagVer");
        for (i = 0; i < (pCase->taglen >> 2U); i++) {
            SEA_MSG("\n%08X %08X", tagSign[i], tagVer[i]);
        }
    } else {
        if (verStat != AES_VERIFY_SUCCESS) {
            SEA_MSG("\n[log]++verStat Fail!!!");
        }
    }
}
} else {
    SEA_MSG("\n[log]++encrypt & decrypt Fail!!!\n");
}

return tmp_err;
}

static ErrStatus aes_gmac_testunit(AES_AAD_TESTLIST *testlist, AES_ENCRYPT_SEL encrypt_sel, uint8_t
aad_mode) {
    ErrStatus tmp_err = SUCCESS;
#ifdef AES_CRYPT_ITEMS_IN_BYTES
    add_t aad[512];
#else
    add_t aad[128];
#endif
}
```

```
#ifdef AES_PROCESS_LATCH_RECOVER_SUPPORT
    aesProcessParameters tmp_latch_buf;
#endif

uint32_t pt[32], ct[32], key[8], iv[64], tagSign[8], tagVer[8], output[32];
uint32_t i, total_len, loops, calcSize, tmp_addr;
AES_AAD_TESTLIST *pCase = testlist;
aesCypherConfigStr testcfg;
AES_VERIFY_STA verStat;

sea_memset_word(pt, 0U, 32U);
sea_memset_word(ct, 0U, 32U);
sea_memset_word(output, 0U, 32U);
sea_memset_word(key, 0U, 8U);
sea_memset_word(iv, 0U, 64U);
sea_memset_word(tagSign, 0U, 8U);
sea_memset_word(tagVer, 0U, 8U);
sea_memset_word((uint32_t *)aad, 0U, 128U);

string2hex((const uint8_t *)pCase->key, (uint8_t *)key, pCase->keylen);
string2hex((const uint8_t *)pCase->iv, (uint8_t *)iv, pCase->ivlen);
string2hex((const uint8_t *)pCase->aad, (uint8_t *)aad, pCase->aadlen);

testcfg.encrypt_sel = encrypt_sel;
testcfg.bitWideMode = pCase->bitwidemode;
testcfg.cryptMode = pCase->cryptmode;
testcfg.keyLocation = (key_t *)&key[0];
testcfg.ivLocation = (iv_t *)&iv[0];
testcfg.ivSize = pCase->ivlen;
if (aad_mode == 1) {
    testcfg.addLocation = (add_t *)&aad[0];
    testcfg.addSize = pCase->aadlen;
} else {
    testcfg.addLocation = (add_t *)0U;
    testcfg.addSize = 0U;
}
testcfg.tagSignBuf = (tag_t *)&tagSign[0];
testcfg.tagVerifyBuf = (tag_t *)&tagVer[0];
testcfg.tagSize = pCase->taglen;
testcfg.tagVerifySta = &verStat;
testcfg.specialMode = AES_SPC_NONE;

string2hex((const uint8_t *)pCase->pt, (uint8_t *)pt, pCase->ptlen);
string2hex((const uint8_t *)pCase->ct, (uint8_t *)ct, pCase->ptlen);
```

```
if (testcfg.encrypt_sel == AES_deCrypt) {
    string2hex((const uint8_t *)pCase->tag, (uint8_t *)tagSign, testcfg.tagSize);
} else {
    string2hex((const uint8_t *)pCase->tag, (uint8_t *)tagVer, testcfg.tagSize);
}

tmp_err = LL_AES_crypt_init((const aesCypherConfigStr *)&testcfg);
if (SUCCESS == tmp_err) {
    if (aad_mode == 0) {
        total_len = pCase->aadlen;
        loops = ((0 != (total_len % 16U)) ? ((total_len >> 4U) + 1U) : (total_len >> 4U));
        for (i = 0; i < loops; i++) {
            WdtFeed();
#ifdef AES_PROCESS_LATCH_RECOVER_SUPPORT
            if ((i == 0) || (i == 3)) {
                SEA_MSG("\n aes_latch recover begin\n");
                aes_process_status_latch(&tmp_latch_buf);
                aes_128_gcm_demo_init(AES_enCrypt);
                aes_128_gcm_demo_crypt_sign();
                aes_process_status_recover(&tmp_latch_buf);
                SEA_MSG("\n aes_latch recover finish\n");
            } else if (i == 1) {
                SEA_MSG("\n aes_latch recover begin\n");
                aes_process_status_latch(&tmp_latch_buf);
                aes_128_gcm_demo_init(AES_deCrypt);
                aes_128_gcm_demo_decrypt_verify();
                aes_process_status_recover(&tmp_latch_buf);
                SEA_MSG("\n aes_latch recover finish\n");
            }
#endif
        }
    }
    calcSize = ((total_len > 16U) ? 16U : total_len);
    total_len -= calcSize;

    tmp_addr = ((uint32_t)aad + (i * 16U));
    tmp_err = LL_AES_update_aad((const add_t *)tmp_addr, calcSize);
    if (SUCCESS != tmp_err) {
        SEA_MSG("\n[log]++update aad Fail!!!");
    }
}
} else {
    SEA_MSG("\n[log]++init Fail!!!\n");
}
if (SUCCESS == tmp_err) {
```

```
tmp_err = LL_AES_gmac((const plainData_t *)aad, pCase->aadlen, CypherDataIsTheLast);
}
(void)LL_AES_crypt_deinit();

if (SUCCESS == tmp_err) {
    if (AES_enCrypt == encrypt_sel) {
        if (0U != memcmp((const uint8_t *)output, (const uint8_t *)ct, pCase->ptlen)) {
            tmp_err = ERROR;
            SEA_MSG("\n[log]++output\tct");
            for (i = 0; i < (pCase->ptlen >> 2U); i++) {
                SEA_MSG("\n%08X %08X", output[i], ct[i]);
            }
        }
    } else {
        if (0U != memcmp((const uint8_t *)output, (const uint8_t *)pt, pCase->ptlen)) {
            tmp_err = ERROR;
            SEA_MSG("\n[log]++output\tpt");
            for (i = 0; i < (pCase->ptlen >> 2U); i++) {
                SEA_MSG("\n%08X %08X", output[i], pt[i]);
            }
        }
    }

    if (0U != memcmp((const uint8_t *)tagVer, (const uint8_t *)tagSign, pCase->taglen)) {
        tmp_err = ERROR;
        SEA_MSG("\n[log]++tagSign\ttagVer");
        for (i = 0; i < (pCase->taglen >> 2U); i++) {
            SEA_MSG("\n%08X %08X", tagSign[i], tagVer[i]);
        }
    } else {
        if (verStat != AES_VERIFY_SUCCESS) {
            SEA_MSG("\n[log]++verStat Fail!!!");
        }
    }
}
} else {
    SEA_MSG("\n[log]++LL_AES_gmac Fail!!!\n");
}

return tmp_err;
}
```

3.3.5. 过程现场保存和恢复

用途描述:

使用 AES 对一帧数据(CASE0)进行加解/解密, 当进行到一半(还未完成所有数据的加密/解密), 若此时

需要插入另外一种 AES 模式 / 另外一帧数据的加密 / 解密 (CASE1)，就需要使用函数 `aes_process_status_latch(&BUF_CASE0)` 将目前正在进行的 AES 加密/解密进程 (CASE0) 进行锁存，然后进行新的 AES 加密/解密 (CASE1)。当 CASE1 完成之后，重新调用函数 `aes_process_status_recover(&BUF_CASE0)`，对 CASE0 的进程进行恢复，继续完成 CASE0 的加密/解密。

在连续数据流通讯过程中，同时存在两种不同的 AES 加密/解密进程。

过程保存的 buffer 需要用户开辟和管理，理论上可以支持任意个 AES 过程的锁存和恢复。

3.3.5.1. 伪代码 demo

3.3.5.1.1. 场景说明

- AES-GCM-CASE0: 从通讯接口不停的接收数据，对数据进行流式 AES-128-GCM 解密；
- AES-GCM-CASE1: 源源不断的产生数据，数据进行 AES-128-GCM 加密后，对外进行传输；
- AES-ECB-CASE2: 临时插入的 AES-256-ECB 解密。

3.3.5.1.2. 伪代码

初始化:

- `aes_crypt_init(AES-GCM-CASE0) /*AES-GCM-CASE0 初始化*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE0) /*AES-GCM-CASE0 过程锁存*/`
- `aes_crypt_init(AES-GCM-CASE1) /*AES-GCM-CASE1 初始化*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程锁存*/`

解密、加密:

- `aes_process_status_recover(&BUF_AES-GCM-CASE0) /*AES-GCM-CASE0 过程恢复*/`
- `aes_decrypt_verify(AES-GCM-CASE0 N*16 字节, is_last = 0) /*完成 N*16 字节 AES-GCM-CASE0 解密*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE0) /*AES-GCM-CASE0 过程锁存*/`
- `aes_process_status_recover(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程恢复*/`
- `aes_crypt_sign(AES-GCM-CASE1 N*16 字节, is_last = 0) /*完成 N*16 字节 AES-GCM-CASE1 加密*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程锁存*/`
- ... /*任意重复*/
- `aes_decrypt_verify(AES-GCM-CASE0 N*16 字节, is_last = 0) /*完成 N*16 字节 AES-GCM-CASE0 解密*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE0) /*AES-GCM-CASE0 过程锁存*/`

临时插入 AES-ECB-CASE2:

- `aes_crypt_init(AES-ECB-CASE2) /*AES-ECB-CASE2 初始化*/`
- `aes_decrypt_verify(AES-ECB-CASE2 N, is_last = 1) /*完成 AES-ECB-CASE2 全部数据解密*/`

恢复 AES-GCM-CASE1，继续加密:

- `aes_process_status_recover(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程恢复*/`
- `aes_crypt_sign(AES-GCM-CASE1 N*16 字节, is_last = 0) /*完成 N*16 字节 AES-GCM-CASE1 加密*/`
- `aes_process_status_latch(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程锁存*/`
- ...

结束 AES-GCM-CASE0 解密，结束 AES-GCM-CASE1 加密:

- `aes_process_status_recover(&BUF_AES-GCM-CASE0) /*AES-GCM-CASE0 过程恢复*/`
- `aes_decrypt_verify(AES-GCM-CASE0 N 字节, is_last = 1) /*结束 AES-GCM-CASE0 解密*/`
- `aes_process_status_recover(&BUF_AES-GCM-CASE1) /*AES-GCM-CASE1 过程恢复*/`
- `aes_crypt_sign(AES-GCM-CASE1 N 字节, is_last = 1) /*结束 AES-GCM-CASE1 加密*/`

4. HASH 库函数

4.1. 函数接口

文件:

- rn8xxx_ll_sea_hash.c
- rn8xxx_ll_sea_hash.h

函数名	函数接口名
HASH 初始化函数	ErrStatus LL_HASH_init(const hashConfigStr* configuration);
HASH 模块关闭函数	ErrStatus LL_HASH_deinit(void);
HASH 计算函数	ErrStatus LL_HASH(const hashData_t* verifyMessage, uint32_t size, CYPHER_DATA_STA is_last, hashData_t*verifyOut);
计算过程现场保存函数(扩展功能, 非必须)	ErrStatus LL_HASH_process_status_latch(hashProcessParameters* latch_buf);
计算过程现场恢复函数(扩展功能, 非必须)	ErrStatus LL_HASH_process_status_recover(const hashProcessParameters* recover_buf);

4.2. 函数说明

4.2.1. 初始化函数

```
ErrStatus LL_HASH_init(const hashConfigStr* configuration);
```

4.2.1.1. configure 结构体

```
typedef struct {
    SEA_SHA_MODE          bitWideMode ; /* SHA-128/224/256/384/512 select*/
    SHA_MSG_SIZE_MODE     msgSize      ; /* NC, not used*/
} hashConfigStr;
```

4.2.2. 计算函数

```
ErrStatus LL_HASH(const hashData_t* verifyMessage, uint32_t size, CYPHER_DATA_STA is_last,
hashData_t*verifyOut);
```

4.2.2.1. 参数描述

参数	说明	适用对象	注意事项
verifyMessage	指向进行哈希计算数据的指针	全部	指针指向地址 byte/word 对齐; 数据存储方式为 Little-Endian。
size	明文长度	全部	单位规定为 bytes。
is_last	计算数据状态配置	全部	当计算为完整数据的最后一次计算时, 需要将 is_last 置 1,其他情况置 0。 is_last = 1,HASH 计算完成后, 将在 verifyOut 里面输出 HASH 结果。
verifyOut	指针指向 HASH 结果输出存放的 buffer 地址	全部	当 is_last = 1 的时候,才进行结果输出。

		指针指向地址 byte/word 对齐; 数据存储方式为 Little-Endian。
--	--	---

4.2.2.2. verifyOut 参数说明

位宽	verifyOut buffer size(byte)
SHA1	160/8
SHA224	224/8
SHA256	256/8
SHA384	384/8
SHA512	512/8

4.3. 验证函数

4.3.1. 伪代码 Demo

说明:

- 函数使用的时候注意判定函数 return 值, 当 return 值=SUCCESS 的时候, 函数操作和数据才有效;
- 整体功能停止使用后, 可使用 hash_deinit()关闭模块时钟。

4.3.1.1. 单次完成完整数据计算

计算 flash 地址 0: (128*1024-1) 数据的哈希值 256 值。

变量 1: ErrStatus ret_error;

变量 2: hashConfigStr hash_cfg == {SEA_SHA_256,{0,0}};

变量 3: uint32_t hash_out[256/8];

1. ret_error = hash_init((const hashConfigStr*)&hash_cfg);
2. ret_error = hash((const hashData_t*)(0), 128*1024, CypherDatasIsTheLast, (hashData_t*)hash_out);

4.3.1.2. 分多次完成完整数据计算

计算 flash 地址 0: (256-1)和 1024: (2*1024-1)数据的哈希值 256 值。

变量 1: ErrStatus ret_error;

变量 2: hashConfigStr hash_cfg == {SEA_SHA_256,{0,0}};

变量 3: uint32_t hash_out[256/8];

1. ret_error = hash_init((const hashConfigStr*)&hash_cfg);
2. ret_error = hash((const hashData_t*)(0), 256, CypherDatasNotTheLast, (hashData_t*)hash_out);
3. ret_error = hash((const hashData_t*)(1024), 1024, CypherDatasIsTheLast, (hashData_t*)hash_out);

4.3.1.3. 一次初始化, 多组数据计算

计算 flash 地址 0: (256-1)数据的哈希 256 值。

计算 flash 地址 1024: (2*1024-1)数据的哈希 256 值。

变量 1: ErrStatus ret_error;

变量 2: hashConfigStr hash_cfg == {SEA_SHA_256,{0,0}};

变量 3: uint32_t hash_out[256/8];

1. ret_error = hash_init((const hashConfigStr*)&hash_cfg);
2. ret_error = hash((const data_t*)(0), 256, CypherDatasIsTheLast, (data_t*)hash_out); //得到 hash 计算结果 1
3. ret_error = hash((const data_t*)(1024), 1024, CypherDatasIsTheLast, (data_t*)hash_out); //得到 hash 计算结

果 2

4.3.2. HASH Demo

4.3.2.1. 数据

标准认证字符串数据（文本数据）

"msg": "E9A9CE8852E2ED71DEC73580D60C18777E91A44F0E07674ED9E827DB5
A42A816C5EE6DEA4F59F85AC9982E34E4D1B241"

"len": 384(bit)

哈希计算结果"md": "4CC18D34EDBA4C024133165B045C8C5AE62ED7D1CC
C42DD1FAE7566B8D1EFEDF04E733F1A2D5E55D048C89181526A03D"

```
#ifndef HASH_CRYPT_ITEMS_IN_BYTES /*地址 U8 字节对齐*/
uint8_t sha_384_reqst_size_384[] = {0xe9,0xa9,0xce,0x88,0x52,0xe2,0xed,0x71,0xde
,0xc7,0x35,0x80,0xd6,0xc0,0x18,0x77,0x7e,0x91,0xa4,0x4f,0xe0,0x7,0x67,0x4e,0xd9,0xe8
,0x27,0xdb,0x5a,0x42,0xa8,0x16,0xc5,0xee,0x6d,0xea,0x4f,0x59,0xf8,0x5a,0xc9,0x98
,0x2e,0x34,0xe4,0xd1,0xb2,0x41};
uint8_t sha_384_expected_size_384[]={0x4c,0xc1,0x8d,0x34,0xed,0xba,0x4c,0x2,0x41
,0x33,0x16,0x5b,0x4,0x5c,0x8c,0x5a,0xe6,0x2e,0xd7,0xd1,0xcc,0xc4,0x2d,0xd1,0xfa
,0xe7,0x56,0x6b,0x8d,0x1e,0xfe,0xdf,0x4,0xe7,0x33,0xf1,0xa2,0xd5,0xe5,0x5d,0x4
,0x8c,0x89,0x18,0x15,0x26,0xa0,0x3d};

uint8_t sha_384_reqst_size_1344[] = {
0x74,0x7E,0x2D,0xB8,0xDC,0x20,0xDF,0x0C,0x73
,0x20,0xA1,0x5B,0xF8,0xB5,0x65,0x7C,0xDC,0x91,0x00,0xAE,0xC7,0xD1,0xD2,0x78,0xEC,0x69
,0xC3,0x55,0x50,0xFF,0x89,0x05,0xED,0x0C,0xF6,0xFD,0xEE,0x8E,0x3D,0x15,0x66,0x58,0xC0
,0x39,0xFD,0xCE,0x4B,0x88,0xFE,0xB4,0x73,0x4C,0x62,0x68,0xAB,0x6B,0x7C,0x1B,0x46,0x1A
,0x36,0x46,0x65,0x33,0x59,0x9F,0x29,0x44,0x3D,0x64,0x30,0x4F,0xDE,0x63,0x17,0x72,0x3C
,0x4E,0x93,0x32,0x83,0xF9,0x9B,0xB6,0x33,0xE3,0xA4,0x16,0x50,0x34,0x61,0x1F,0xDE,0x1C
,0x78,0x7D,0xCB,0xFD,0x88,0x91,0x00,0xCF,0x81,0x9A,0x6A,0xA7,0x2F,0x07,0x7F,0xFC,0x9F
,0xDD,0x3A,0x24,0x36,0xAE,0xC4,0x31,0x30,0xE1,0x30,0x2A,0x13,0xBF,0xFE,0x9B,0xF6,0x0D
,0x6A,0x96,0x6E,0xDC,0x2C,0xCE,0x57,0xBA,0xA8,0x60,0xE7,0x64,0xC5,0xD5,0x41,0x64,0xF4
,0x60,0xBA,0xD4,0x29,0xE2,0x14,0x3A,0xD9,0xB4,0x60,0x58,0x89,0x29,0xFB,0x52,0x0B,0xF4
,0xDC,0xC0,0x60,0x93,0x3E,0xF5};
uint8_t sha_384_expected_size_1344[]={0x1D,0x16,0x8F,0x23,0x91,0x6C,0xE3,0x9B,0xD5
,0xE4,0x7D,0xAA,0xDD,0x34,0xCC,0xAE,0xB5,0x2F,0x2E,0x4F,0x61,0x7D,0xB0,0xA1,0x67
,0x50,0xA7,0x9A,0x0A,0xE0,0x5A,0x70,0x87,0x0E,0xCF,0x9B,0x2F,0x81,0x9A,0xFD,0x51
,0x7B,0x44,0x0D,0x2F,0xC3,0x95,0xE0};

uint8_t sha_256_reqst_size_24[] = {0xEC, 0x7E, 0xEF};
uint8_t sha_256_expected_size_24[]={0xF4,0x34,0xDE,0x29,0x63,0x64,0x0D,0x0A,0x96
,0x3F,0x46,0xD9,0x68,0xAA,0x88,0x01,0x31,0xDB,0x98,0x20,0xFB,0xC9,0xFE,0x4F,0x0E
,0x0A,0xFE,0xCD,0x85,0xF8,0xB5,0x7A};
#else
```



```
#if defined ( __CC_ARM )/*地址 U32 字节对齐*/
__align(4) uint8_t sha_384_reqst_size_384[] = {0xe9,0xa9,0xce,0x88,0x52,0xe2,0xed
,0x71,0xde,0xc7,0x35,0x80,0xd6,0xc,0x18,0x77,0x7e,0x91,0xa4,0x4f,0xe,0x7,0x67,0x4e
,0xd9,0xe8,0x27,0xdb,0x5a,0x42,0xa8,0x16,0xc5,0xee,0x6d,0xea,0x4f,0x59,0xf8,0x5a
,0xc9,0x98,0x2e,0x34,0xe4,0xd1,0xb2,0x41};
__align(4) uint8_t sha_384_expected_size_384[]={0x4c,0xc1,0x8d,0x34,0xed,0xba,0x4c
,0x2,0x41,0x33,0x16,0x5b,0x4,0x5c,0x8c,0x5a,0xe6,0x2e,0xd7,0xd1,0xcc,0xc4,0x2d,0xd1
,0xfa,0xe7,0x56,0x6b,0x8d,0x1e,0xfe,0xdf,0x4,0xe7,0x33,0xf1,0xa2,0xd5,0xe5,0x5d,0x4
,0x8c,0x89,0x18,0x15,0x26,0xa0,0x3d};
__align(4) uint8_t sha_384_reqst_size_1344[] = {
0x74,0x7E,0x2D,0xB8,0xDC,0x20,0xDF,0x0C,0x73
,0x20,0xA1,0x5B,0xF8,0xB5,0x65,0x7C,0xDC,0x91,0x00,0xAE,0xC7,0xD1,0xD2,0x78,0xEC,0x69
,0xC3,0x55,0x50,0xFF,0x89,0x05,0xED,0x0C,0xF6,0xFD,0xEE,0x8E,0x3D,0x15,0x66,0x58,0xC0
,0x39,0xFD,0xCE,0x4B,0x88,0xFE,0xB4,0x73,0x4C,0x62,0x68,0xAB,0x6B,0x7C,0x1B,0x46,0x1A
,0x36,0x46,0x65,0x33,0x59,0x9F,0x29,0x44,0x3D,0x64,0x30,0x4F,0xDE,0x63,0x17,0x72,0x3C
,0x4E,0x93,0x32,0x83,0xF9,0x9B,0xB6,0x33,0xE3,0xA4,0x16,0x50,0x34,0x61,0x1F,0xDE,0x1C
,0x78,0x7D,0xCB,0xFD,0x88,0x91,0x00,0xCF,0x81,0x9A,0x6A,0xA7,0x2F,0x07,0x7F,0xFC,0x9F
,0xDD,0x3A,0x24,0x36,0xAE,0xC4,0x31,0x30,0xE1,0x30,0x2A,0x13,0xBF,0xFE,0x9B,0xF6,0x0D
,0x6A,0x96,0x6E,0xDC,0x2C,0xCE,0x57,0xBA,0xA8,0x60,0xE7,0x64,0xC5,0xD5,0x41,0x64,0xF4
,0x60,0xBA,0xD4,0x29,0xE2,0x14,0x3A,0xD9,0xB4,0x60,0x58,0x89,0x29,0xFB,0x52,0x0B,0xF4
,0xDC,0xC0,0x60,0x93,0x3E,0xF5};
__align(4) uint8_t sha_384_expected_size_1344[]={0x1D,0x16,0x8F,0x23,0x91,0x6C,0xE3
,0x9B,0xD5,0xE4,0x7D,0xAA,0xDD,0x34,0xCC,0xAE,0xB5,0x2F,0x2E,0x4F,0x61,0x7D,0xB0,0xA1
,0x67,0x50,0xA7,0x9A,0x0A,0xE0,0x5A,0x70,0x87,0x0E,0xCF,0x9B,0x2F,0x81,0x9A,0xFD,0x51
,0x7B,0x44,0x0D,0x2F,0xC3,0x95,0xE0};
__align(4) uint8_t sha_256_reqst_size_24[] = {0xEC, 0x7E, 0xEF};
__align(4) uint8_t sha_256_expected_size_24[]={0xF4,0x34,0xDE,0x29,0x63,0x64,0x0D
,0x0A,0x96,0x3F,0x46,0xD9,0x68,0xAA,0x88,0x01,0x31,0xDB,0x98,0x20,0xFB,0xC9,0xFE
,0x4F,0x0E,0x0A,0xFE,0xCD,0x85,0xF8,0xB5,0x7A};
#else /*defined ( __ICCARM__ )*/
#pragma pack(4)
uint8_t sha_384_reqst_size_384[] = {0xe9,0xa9,0xce,0x88,0x52,0xe2,0xed,0x71,0xde
,0xc7,0x35,0x80,0xd6,0xc,0x18,0x77,0x7e,0x91,0xa4,0x4f,0xe,0x7,0x67,0x4e,0xd9
,0xe8,0x27,0xdb,0x5a,0x42,0xa8,0x16,0xc5,0xee,0x6d,0xea,0x4f,0x59,0xf8,0x5a,0xc9
,0x98,0x2e,0x34,0xe4,0xd1,0xb2,0x41};
uint8_t sha_384_expected_size_384[]={0x4c,0xc1,0x8d,0x34,0xed,0xba,0x4c,0x2,0x41
,0x33,0x16,0x5b,0x4,0x5c,0x8c,0x5a,0xe6,0x2e,0xd7,0xd1,0xcc,0xc4,0x2d,0xd1,0xfa
,0xe7,0x56,0x6b,0x8d,0x1e,0xfe,0xdf,0x4,0xe7,0x33,0xf1,0xa2,0xd5,0xe5,0x5d,0x4
,0x8c,0x89,0x18,0x15,0x26,0xa0,0x3d};
uint8_t sha_384_reqst_size_1344[] = {
0x74,0x7E,0x2D,0xB8,0xDC,0x20,0xDF,0x0C,0x73
,0x20,0xA1,0x5B,0xF8,0xB5,0x65,0x7C,0xDC,0x91,0x00,0xAE,0xC7,0xD1,0xD2,0x78,0xEC,0x69
,0xC3,0x55,0x50,0xFF,0x89,0x05,0xED,0x0C,0xF6,0xFD,0xEE,0x8E,0x3D,0x15,0x66,0x58,0xC0
```

```
,0x39,0xFD,0xCE,0x4B,0x88,0xFE,0xB4,0x73,0x4C,0x62,0x68,0xAB,0x6B,0x7C,0x1B,0x46,0x1A  
,0x36,0x46,0x65,0x33,0x59,0x9F,0x29,0x44,0x3D,0x64,0x30,0x4F,0xDE,0x63,0x17,0x72,0x3C  
,0x4E,0x93,0x32,0x83,0xF9,0x9B,0xB6,0x33,0xE3,0xA4,0x16,0x50,0x34,0x61,0x1F,0xDE,0x1C  
,0x78,0x7D,0xCB,0xFD,0x88,0x91,0x00,0xCF,0x81,0x9A,0x6A,0xA7,0x2F,0x07,0x7F,0xFC,0x9F  
,0xDD,0x3A,0x24,0x36,0xAE,0xC4,0x31,0x30,0xE1,0x30,0x2A,0x13,0xBF,0xFE,0x9B,0xF6,0x0D  
,0x6A,0x96,0x6E,0xDC,0x2C,0xCE,0x57,0xBA,0xA8,0x60,0xE7,0x64,0xC5,0xD5,0x41,0x64,0xF4  
,0x60,0xBA,0xD4,0x29,0xE2,0x14,0x3A,0xD9,0xB4,0x60,0x58,0x89,0x29,0xFB,0x52,0x0B,0xF4  
,0xDC,0xC0,0x60,0x93,0x3E,0xF5};
```

```
uint8_t sha_384_expected_size_1344[] = {0x1D,0x16,0x8F,0x23,0x91,0x6C,0xE3,0x9B,0xD5  
,0xE4,0x7D,0xAA,0xDD,0x34,0xCC,0xAE,0xB5,0x2F,0x2E,0x4F,0x61,0x7D,0xB0,0xA1,0x67  
,0x50,0xA7,0x9A,0x0A,0xE0,0x5A,0x70,0x87,0x0E,0xCF,0x9B,0x2F,0x81,0x9A,0xFD,0x51  
,0x7B,0x44,0x0D,0x2F,0xC3,0x95,0xE0};
```

```
uint8_t sha_256_request_size_24[] = {0xEC, 0x7E, 0xEF};
```

```
uint8_t sha_256_expected_size_24[] = {0xF4,0x34,0xDE,0x29,0x63,0x64,0x0D,0x0A,0x96  
,0x3F,0x46,0xD9,0x68,0xAA,0x88,0x01,0x31,0xDB,0x98,0x20,0xFB,0xC9,0xFE,0x4F,0x0E  
,0x0A,0xFE,0xCD,0x85,0xF8,0xB5,0x7A};
```

```
#pragma pack()
```

```
#endif
```

```
#endif
```

4.3.2.2. 单次完成 hash 计算

```
/*单次完成 hash 计算*/
```

```
void hash_384_demo_single_calc(void)
```

```
{  
    ErrStatus ret_error;  
    hashConfigStr hash_cfg = {SEA_SHA_384, {sizeof(sha_384_request_size_1344), 0}};  
    uint32_t hash_out[384/8];  
    ret_error = hash_init((const hashConfigStr*)&hash_cfg);  
    if(ret_error != SUCCESS){  
        printf("\n-hash_init return error\n");  
        return;  
    }  
    ret_error = hash((const data_t*)(sha_384_request_size_1344)  
    , sizeof(sha_384_request_size_1344), CypherDatasIsTheLast, (data_t*)hash_out);  
    if(ret_error != SUCCESS){  
        printf("\n-hash return error\n");  
        return;  
    }  
    if(0 == memcmp((const uint8_t*)sha_384_expected_size_1344  
    , (const uint8_t*)hash_out, sizeof(sha_384_expected_size_1344))){  
        printf("\n hash out compare pass \n");  
    }else{  
        printf("\n hash out compare fail \n");  
    }  
}
```

```
}  
}
```

4.3.2.3. 分多次完成 hash 计算

/*分多次完成 hash 计算*/

```
void hash_384_demo_multi_calc(void)  
{  
    ErrStatus ret_error;  
    hashConfigStr hash_cfg = {SEA_SHA_384, {sizeof(sha_384_request_size_1344), 0}};  
    uint32_t hash_out[384/8];  
    ret_error = hash_init((const hashConfigStr*) &hash_cfg);  
    if(ret_error != SUCCESS){  
        printf("\n-hash_init return error\n");  
        return;  
    }  
    ret_error = hash((const data_t*)(sha_384_request_size_1344), 128  
/*when is_last = CypherDatanotTheLast, size = 128*N*/  
    , CypherDatanotTheLast, (data_t*)hash_out);  
    if(ret_error != SUCCESS){  
        printf("\n-hash return error\n");  
        return;  
    }  
    ret_error = hash((const data_t*)&sha_384_request_size_1344[128/sizeof(uint32_t)]  
    , sizeof(sha_384_request_size_1344) - 128, CypherDatanotTheLast  
    , (data_t*)&hash_out[128/sizeof(uint32_t)]);  
    if(ret_error != SUCCESS){  
        printf("\n-hash return error\n");  
        return;  
    }  
    if(0 == memcmp((const uint8_t*)sha_384_expected_size_1344  
    , (const uint8_t*)hash_out, sizeof(sha_384_expected_size_1344))){  
        printf("\n hash out compare pass \n");  
    }else{  
        printf("\n hash out compare fail \n");  
    }  
}
```

4.3.2.4. 同类型 SHA 多组数据连续计算

/*单次初始化，同类型多组数据分别完成 hash 计算*/

```
void hash_384_demo_singleInit_multiCalc(void)  
{  
    ErrStatus ret_error;  
    hashConfigStr hash_cfg = {SEA_SHA_384, {0, 0}};  
    uint32_t hash_out[384/8];
```

```
ret_error = hash_init((const hashConfigStr* )&hash_cfg);
if(ret_error != SUCCESS){
    printf("\n-hash_init return error\n");
    return;
}
//完成第一组， 1344byte 长度数据的 hash 计算
ret_error = hash((const data_t*)(sha_384_reqst_size_1344)
, sizeof(sha_384_reqst_size_1344), CypherDatasIsTheLast, (data_t*)hash_out);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}
if(0 == memcmp((const uint8_t*)sha_384_expected_size_1344
, (const uint8_t*)hash_out, sizeof(sha_384_expected_size_1344))){
    printf("\n hash out1 compare pass \n");
}else{
    printf("\n hash out1 compare fail \n");
}
//完成第二组， 1344byte 长度数据的 hash 计算
ret_error = hash((const data_t*)(sha_384_reqst_size_384)
, sizeof(sha_384_reqst_size_384), CypherDatasIsTheLast, (data_t*)hash_out);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}
if(0 == memcmp((const uint8_t*)sha_384_expected_size_384
, (const uint8_t*)hash_out, sizeof(sha_384_expected_size_384))){
    printf("\n hash out2 compare pass \n");
}else{
    printf("\n hash out2 compare fail \n");
}
}
```

4.3.2.5. 任意数据长度计算

```
void hash_384_demo_rand_size_calc(void)
{
    ErrStatus ret_error;
    hashConfigStr hash_cfg = {SEA_SHA_384, {sizeof(sha_384_reqst_size_1344),0}};
    uint32_t hash_out[384/8];
    #ifdef HASH_CRYPT_ITEMS_IN_BYTES
    uint8_t rand_size[3] = {1,2,3};
    #else
    uint8_t rand_size[4] = {4,8,12};
    #endif
}
```

```
ret_error = hash_init((const hashConfigStr* )&hash_cfg);
if(ret_error != SUCCESS){
    printf("\n-hash_init return error\n");
    return;
}

ret_error = hash((const hashData_t*)(sha_384_request_size_1344), rand_size[0],
CypherDatanotTheLast, (hashData_t *)0);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}

ret_error = hash((const hashData_t*)&sha_384_request_size_1344[rand_size[0]],
rand_size[1], CypherDatanotTheLast, (hashData_t *)0);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}

ret_error = hash((const hashData_t*)&sha_384_request_size_1344[rand_size[0]
+ rand_size[1]], rand_size[2], CypherDatanotTheLast, 0);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}

ret_error = hash((const hashData_t*)&sha_384_request_size_1344[rand_size[0]
+ rand_size[1] + rand_size[2]], sizeof(sha_384_request_size_1344)
- rand_size[0] - rand_size[1] - rand_size[2], CypherDatanotTheLast
, (hashData_t *)hash_out);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}
if(0 == memcmp((const uint8_t*)sha_384_expected_size_1344, (const uint8_t*)hash_out
, sizeof(sha_384_expected_size_1344))){
    printf("\n hash out compare pass \n");
} else {
    printf("\n hash out compare fail \n");
}
}

void hash_256_demo_rand_size_calc(void)
{
```

```
ErrStatus ret_error;
hashConfigStr hash_cfg = {SEA_SHA_256, {sizeof(sha_256_request_size_24), 0}};
uint32_t hash_out[384/8];
#ifdef HASH_CRYPT_ITEMS_IN_BYTES
uint8_t rand_size[2] = {1, 2};
#else
uint8_t rand_size[2] = {0, 3};
#endif
ret_error = hash_init((const hashConfigStr*)&hash_cfg);
if(ret_error != SUCCESS){
    printf("\n-hash_init return error\n");
    return;
}
ret_error = hash((const hashData_t*)(sha_256_request_size_24), rand_size[0],
    CypherDatasNotTheLast, (hashData_t*)0);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}

ret_error = hash((const hashData_t*)&sha_256_request_size_24[rand_size[0]],
    rand_size[1], CypherDataIsTheLast, (hashData_t*)hash_out);
if(ret_error != SUCCESS){
    printf("\n-hash return error\n");
    return;
}

if(0 == memcmp((const uint8_t*)sha_256_expected_size_24, (const uint8_t*)hash_out,
    sizeof(sha_256_expected_size_24))){
    printf("\n hash out compare pass \n");
} else {
    printf("\n hash out compare fail \n");
}
}
```

4.3.2.6. 总 demo 函数接口

```
void hash_demo_test(void)
{
    WdtFeed();
    printf("\n -----hash test begin ----- \n");
    //单次完成 hash 计算
    hash_384_demo_single_calc();
    //分多次完成 hash 计算
    hash_384_demo_multi_calc();
}
```

```
//单次初始化, 多组数据分别完成 hash 计算
hash_384_demo_singleInit_mulitiCalc();

//随意数据长度,完成 hash 计算
hash_384_demo_rand_size_calc();
hash_256_demo_rand_size_calc();

printf("\n -----hash test finish ----- \n");

}
```

4.3.3. 过程现场保存和恢复

用途描述:

1. 使用 HASH 对一帧数据(CASE0)进行计算的时候, 在进行到一半的时候, 还未完成所有数据的计算, 中间就需要插入(CASE1)另外一种 HASH 模式/另外一帧数据的计算, 则此时就需要使用函数 `hash_process_status_latch(&BUF_CASE0)`将目前正在进行的 HASH 计算进程(CASE0)进行锁存, 然后进行新的 HASH 计算 (CASE1)。当 CASE1 完成之后, 重新调用函数 `hash_process_status_recover(&BUF_CASE0)`,对 CASE0 的进程进行恢复。然后继续完成 CASE0 的计算;
2. 在连续数据流通讯过程中, 同时存在两种不同的 HASH 计算进程;
3. 过程保存的 buffer 需要用户开辟和管理, 理论上可以支持任意个 HASH 过程的锁存和恢复。

4.3.3.1. 伪代码 demo

4.3.3.1.1. 场景说明

- SHA256-CASE0: 从通讯接口不停的接收数据, 对数据进行流式 SHA256 计算;
- SHA384-CASE1: 源源不断的产生数据, 数据进行 SHA384 计算, 对外进行传输;
- SHA384-CASE2: 临时插入的 SHA384 计算。

4.3.3.1.2. 伪代码

初始化:

- `hash_init(SHA256-CASE0)` /*SHA256-CASE0 初始化*/
- `hash_process_status_latch(&BUF_SHA256-CASE0)`/*SHA256-CASE0 过程锁存*/
- `hash_init(SHA384-CASE1)` /*SHA384-CASE1 初始化*/
- `hash_process_status_latch(&BUF_SHA384-CASE1)`/*SHA384-CASE1 过程锁存*/

HASH 计算:

- `hash_process_status_recover(&BUF_SHA256-CASE0)`/*SHA256-CASE0 过程恢复*/
- `hash(SHA256-CASE0 N 字节, is_last = 0)`/*完成 N 字节 SHA256-CASE0 计算*/
- `hash_process_status_latch(&BUF_SHA256-CASE0)`/*SHA256-CASE0 过程锁存*/
- `hash_process_status_recover(&BUF_SHA384-CASE1)`/*SHA384-CASE1 过程恢复*/
- `hash(SHA384-CASE1 N 字节, is_last = 0)`/*完成 N 字节 SHA384-CASE1 计算*/
- `hash_process_status_latch(&BUF_SHA384-CASE1)`/*SHA384-CASE1 过程锁存*/
- ...
- `hash_process_status_recover(&BUF_SHA256-CASE0)`/*SHA256-CASE0 过程恢复*/

- hash(SHA256-CASE0 N 字节, is_last = 0)/*完成 N 字节 SHA256-CASE0 计算*/
- hash_process_status_latch(&BUF_SHA256-CASE0)/*SHA256-CASE0 过程锁存*/

临时插入 SHA384-CASE2:

- hash_init(SHA384-CASE2) /*SHA384-CASE2 初始化*/
- hash(SHA384-CASE2 N 字节, is_last = 1)/*完成 SHA384-CASE2 全部数据计算*/
- 恢复 SHA384-CASE1, 继续计算:
- hash_process_status_recover(&BUF_SHA384-CASE1)/*SHA384-CASE1 过程恢复*/
- hash(SHA384-CASE1 N 字节, is_last = 0)/*完成 N 字节 SHA384-CASE1 计算*/
- hash_process_status_latch(&BUF_SHA384-CASE1)/*SHA384-CASE1 过程锁存*/
- ...

结束 SHA256-CASE0 计算, 结束 SHA384-CASE1 计算:

- hash_process_status_recover(&BUF_SHA256-CASE0)/*SHA256-CASE0 过程恢复*/
- hash(SHA256-CASE0 N 字节, is_last = 1)/*结束 SHA256-CASE0 计算*/
- hash_process_status_recover(&BUF_SHA384-CASE1)/*SHA384-CASE1 过程恢复*/
- hash(SHA384-CASE1 N 字节, is_last = 1)/*结束 SHA384-CASE1 计算*/

5. ECC 库函数

5.1. 函数接口

接口文件：

- rn8xxx_ll_sea_ecc.c
- rn8xxx_ll_sea_ecc.h

函数名	函数接口名
公私密钥对生成函数	ErrStatus LL_ECC_keygen(const uECC_Curve_t *curve, ECCdata_t *_Q, ECCdata_t *_pkey);
公钥验证函数	ErrStatus LL_ECC_keyver(const uECC_Curve_t *curve, const ECCdata_t *_Q, PassFlag *ver_flag);
ECDSA 签名生成函数 I (公私钥为返回值)	ErrStatus LL_ECC_sign_gen(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, ECCdata_t *_Q, ECCdata_t *_pkey, ECCdata_t *_r, ECCdata_t *_s);
ECDSA 签名验证函数	ErrStatus LL_ECC_sign_ver(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, const ECCdata_t *_Q, const ECCdata_t *_r, const ECCdata_t *_s, PassFlag *ver_flag);
ECDSA 签名生成函数 II (私钥是输入参数)	ErrStatus LL_ECC_specify_pkey_sign_gen(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, ECCdata_t *_Q, ECCdata_t *_pkey, ECCdata_t *_r, ECCdata_t *_s);
公钥生成函数	ErrStatus LL_ECC_public_keygen(const uECC_Curve_t *curve, ECCdata_t *_Q, const ECCdata_t *_pkey);
密钥协商函数	ErrStatus LL_ECC_DHkeygen_x(const uECC_Curve_t *curve, const ECCdata_t *_Q, const ECCdata_t *_pkey, ECCdata_t *_DHkey);
带有非压缩编码格式的公钥生成函数	ErrStatus LL_ECC_public_keygen_uncompressed(const uECC_Curve_t *curve, ECCdata_t *_Q, const ECCdata_t *_pkey);

5.2. 函数说明

5.2.1. 公私密钥对生成函数

ErrStatus LL_ECC_keygen(const uECC_Curve_t *curve, ECCdata_t *_Q, ECCdata_t *_pkey);
完成 ECC 公钥和私钥的生成。

5.2.1.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Q	用户可开辟空间存储公钥	全部	空间大小至少为 2 倍曲线位宽 (不足 1 字节按 1 字节计), 低地址存 Qx, 高地址存储 Qy。 指针指向地址 byte/word 对齐。
_pkey	用户可开辟空间	全部	空间大小至少为曲线位宽 (不足 1 字节按 1 字节计)。

	存储私钥		指针指向地址 byte/word 对齐。
--	------	--	----------------------

5.2.1.2. 返回值说明

返回值	说明
SUCCESS	无异常错误
ERROR	异常错误： 0x21 硬件超时等待 0x24 循环超时等待

5.2.2. 公钥验证函数

ErrStatus LL_ECC_keyver(const uECC_Curve_t *curve, const ECCdata_t *_Q, PassFlag *ver_flag);
完成 ECC 公钥的验证。

5.2.2.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Q	待验证的公钥	全部	指针类型数据，指向 Qx 首地址。 指针指向地址 byte/word 对齐。
ver_flag	验证通过与否标志指针	全部	值为 0，代表通过，否则不通过。

5.2.2.2. 返回值说明

返回值	说明
SUCCESS	验证通过
ERROR	验证不通过

5.2.3. ECDSA 签名生成函数 I

ErrStatus LL_ECC_sign_gen(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, ECCdata_t *_Q, ECCdata_t *_pkey, ECCdata_t *_r, ECCdata_t *_s);

完成 ECDSA 签名数对的生成，**输入参数为 Hm，输出参数为公私钥对和签名对。**

注意：在打开地址对齐的宏（见 2.2.3）时，参数输入均为大端字节数组。宏关闭时，参数输入均为小端 words 对齐数组。该函数新增参数 Hm_len，用于字节支持。

5.2.3.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Hm	将输入的消息进行 HASH 加密，得到的 HASH 结果	全部	用户至少开辟曲线位宽长度空间。 指针指向地址 byte/word 对齐。

Hm_len	明文长度	全部	字节长度。
_Q	用户可开辟空间存储公钥	全部	空间大小至少为 2 倍曲线位宽（不足 1 字节按 1 字节计），低地址存 Qx，高地址存储 Qy。 指针指向地址 byte/word 对齐。
_pkey	用户可开辟空间存储私钥	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。
_r	用户可开辟空间存储签名数对之一	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。
_s	用户可开辟空间存储签名数对之一	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。

5.2.3.2. 返回值说明

返回值	说明
SUCCESS	无异常错误
ERROR	异常错误： ● 0x21 硬件超时等待 ● 0x24 循环超时等待

5.2.4. ECDSA 签名验证函数

ErrStatus LL_ECC_sign_ver(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, const ECCdata_t *_Q, const ECCdata_t *_r, const ECCdata_t *_s, PassFlag *ver_flag);

完成 ECDSA 签名的验证，输入参数为 Hm、Q 和签名对，输出参数为 ver_flag 标志。

注意：在打开地址对齐的宏（见 2.2.3）时，参数输入均为大端字节数组。宏关闭时，参数输入均为小端 words 对齐数组。该函数新增参数 Hm_len，用于字节支持。

5.2.4.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Hm	将输入的消息进行 HASH 加密，得到的 HASH 结果	全部	用户至少开辟曲线位宽长度空间。 指针指向地址 byte/word 对齐。
Hm_len	明文长度	全部	字节长度。
_Q	待验证的公钥	全部	指针类型数据，指向 Qx 首地址。 指针指向地址 byte/word 对齐。
_r	待验证的签名数对之一	全部	指针类型数据，指向 r 首地址。 指针指向地址 byte/word 对齐。
_s	待验证的签名数对之一	全部	指针类型数据，指向 s 首地址。 指针指向地址 byte/word 对齐。
ver_flag	验证通过与否标志指针	全部	值为 0，代表通过，否则不通过。

5.2.4.2. 返回值说明

返回值	说明
SUCCESS	验证通过
ERROR	验证不通过

5.2.5. ECDSA 生成签名函数 II

ErrStatus LL_ECC_specify_pkey_sign_gen(const uECC_Curve_t *curve, const ECCdata_t *_Hm, const uint32_t Hm_len, ECCdata_t *_Q, ECCdata_t *_pkey, ECCdata_t *_r, ECCdata_t *_s);

完成 ECDSA 签名的生成，该函数的输入参数为 **Hm** 和 **pkey**，输出参数为公钥 **Q** 和签名对。

注意：在打开地址对齐的宏（见 2.2.3）时，参数输入均为大端字节数组。宏关闭时，参数输入均为小端 words 对齐数组。该函数新增参数 **Hm_len**，用于字节支持。

5.2.5.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Hm	将输入的消息进行 HASH 加密，得到的 HASH 结果	全部	用户至少开辟曲线位宽长度空间。 指针指向地址 byte/word 对齐。
Hm_len	明文长度	全部	字节长度。
_Q	用户可开辟空间存储公钥	全部	空间大小至少为 2 倍曲线位宽（不足 1 字节按 1 字节计）， 低地址存 Qx，高地址存储 Qy。 指针指向地址 byte/word 对齐。
_pkey	用户传入私钥	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。
_r	用户可开辟空间存储签名数对之一	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。
_s	用户可开辟空间存储签名数对之一	全部	空间大小至少为曲线位宽（不足 1 字节按 1 字节计）。 指针指向地址 byte/word 对齐。

5.2.5.2. 返回值说明

返回值	说明
SUCCESS	无异常错误
ERROR	异常错误： ● 0x21 硬件超时等待 ● 0x24 循环超时等待

5.2.6. 公钥生成函数

ErrStatus LL_ECC_public_keygen(const uECC_Curve_t *curve, ECCdata_t *_Q, const ECCdata_t *_pkey);

完成 ECDSA 公钥的生成。

5.2.6.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Q	待生成的公钥	全部	指针类型数据，指向 Qx 首地址。 指针指向地址 byte/word 对齐。

5.2.6.2. 返回值说明

返回值	说明
SUCCESS	公钥生成成功
ERROR	公钥生成失败

5.2.7. 密钥协商函数

ErrStatus LL_ECC_DHkeygen_x(const uECC_Curve_t *curve, const ECCdata_t *_Q, const ECCdata_t *_pkey, ECCdata_t *_DHkey);

生成协商的公共密钥，输出 x 坐标用于 A 和 B 双方 check。

5.2.7.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Q	密钥协商指定的公钥	全部	指针类型数据，指向 Qx 首地址。 指针指向地址 byte/word 对齐。
_pkey	密钥协商指定的私钥	全部	指针类型数据，指向 pkey 首地址。 指针指向地址 byte/word 对齐。
_DHkey	输出的 x 坐标	全部	指针类型数据，指向 DHkey_x 首地址。 指针指向地址 byte/word 对齐。

5.2.7.2. 返回值说明

返回值	说明
SUCCESS	生成密钥协商公钥成功
ERROR	生成密钥协商公钥失败

5.2.8. 带有非压缩编码格式的公钥生成函数

ErrStatus LL_ECC_public_keygen_uncompressed (const uECC_Curve_t* curve, uint8_t *Q, const uint32_t pkey[]);

生成带有编码格式的公钥，仅支持非压缩格式。

5.2.8.1. 参数说明

参数	说明	适用对象	注意事项
curve	用户可选择曲线类型	全部	Curve_NIST_P192/Curve_NIST_P224/Curve_NIST_P256/ Curve_NIST_P384/Curve_NIST_P521/ Curve_Brainpool_256/ Curve_Brainpool_384/ Curve_Brainpool_512
_Q	待生成的公钥	全部	指针类型数据, 为 bytes 数组, 带有编码格式的公钥对(编 码头+Qx+Qy)。 指针指向地址 byte/word 对齐。
_pkey	用于生成公钥对的私钥	全部	指针类型数据, 指向 pkey 首地址。 指针指向地址 byte/word 对齐。

5.2.8.2. 返回值说明

返回值	说明
SUCCESS	生成公钥成功
ERROR	生成公钥成功

5.3. 验证函数

5.3.1. 自签名自校验函数

5.3.1.1. 函数说明

1. 完成曲线 NIST_P256 或者 NIST_P384 ECDSA 签名与验证;
2. 将 HASH (SHA-256 或者 SHA-384) 结果打印出来;
3. 将 ECDSA 签名结果打印出来, 包括公钥 Q、私钥 pkey、签名数对 (r, s);
4. 将 ECDSA 校验结果打印出来, 通过: PASS, 不通过: FAIL。

函数原型:

```
void ECC_NISTP256_test(void);
void ECC_NISTP384_test(void);
```

5.3.1.2. 函数调用

- void hash_once(const unsigned char* message, const hashConfigStr* sha_cfg, uint32_t* Hm,uint32_t Hm_len)

完成一次 HASH 运算。

message--输入消息, 目前是字符串输入;
sha_cfg --配置选择 SHA 类型, 消息长度;
Hm --用户根据 SHA 类型开辟 HASH 结果存储空间大小;
Hm_len --HASH 结果长度;

步骤:

1. 初始化 hash 时钟、寄存器等;
2. 计算 hash 值;

3. hash 结果输出。

注：如果要输入到 ECC 模块进行后续计算，则必须进行字节反序，如：**{0x76543210, 0xFEDCBA98}→{0x98BADCFE, 0x10325476}**，大括号左边 32 bits 为低字地址。

● static void string2hex(const uint8_t* str, uint8_t* number, uint32_t len)

进行字符串转 HEX 操作。

str --字符串输入；

number --HEX 数值输出；

len --每 2 个字符对应 1 个字节，字符串转换后对应的字节长度；

注：**"12345678"→{0x12, 0x34, 0x56, 0x78}**，大括号左边 8 bits 为低字节地址。

5.3.1.3. 函数流程

1. 调用 hash_once 函数，计算 hash 值，并打印 hash 结果；
2. 调用库函数 ecc_sign_gen，进行公私密钥对（Q, pkey）、签名数对（r, s）生成，并打印；
3. 调用库函数 ecc_sign_ver，进行公钥、签名验证，输出验证结果；

5.3.2. NIST 标准数据验证函数

5.3.2.1. 函数说明

1. 将 NIST 数据文件字符串数据转换为库函数接口匹配 HEX 数据；
2. 进行 HASH 运算；
3. 对公钥、签名数对（r, s）进行校验，输出结果：通过：PASS，不通过：FAIL。

函数原型：

void ECDSA_Ver_NIST256(void)

void ECDSA_Ver_NIST384(void)

5.3.2.2. NIST 标准数据说明

```
NIST-P256
"algorithm": "ECDSA",
"mode": "sigVer",
{
    "tcid": 64,
    "message": "97B70AF1C5DA8483A0040345A47DF796060E332B1C76D1EC3091633D0CB66",
    "qx": "20D3EDB582A7A2AD678EEAF3D6BC265DCE1D50FC2ED062B0787768989E40C488",
    "qy": "C1EE738BB1EA38E13446F63D4BA66F4B051A55F21C37D8BD48EE4DC2165D2211",
    "r": "585FD4B43C2B027430E202BDFB92574166E5EA563D5174FB861C6946CA40DCC4",
    "s": "956478653FDA85694A603C4557378A94CD73E261994E5FA5E162220C6FF5B5EA",
    "testPassed": true
}
```

1. “tcid”：认证数据序号，无意义；
2. “message”：认证消息，长度为 1024 bits；
3. “qx”：公钥 x 坐标；“qy”：公钥 y 坐标；
4. “r”，“s”签名数对；
5. “testPassed”签名校验通过标志，返回 true 或者 false

注：NIST 认证参考数据存储在《NIST_STANDARD》文件中，库函数均通过该文件数据验证。

5.3.2.3. 函数中 NIST 数据接口

```
const unsigned char message[] = "97B70AF1C5DA8483A0040345A47DF796060E332B1C76D1EC3091633D0CB66";
const char Qx[] = "20D3EDB582A7A2AD678EEAF3D6BC265DCE1D50FC2ED062B0787768989E40C488";
const char Qy[] = "C1EE738BB1EA38E13446F63D4BA66F4B051A55F21C37D8BD48EE4DC2165D2211";
const char R[] = "585FD4B43C2B027430E202BDFB92574166E5EA563D5174FB861C6946CA40DCC4";
const char S[] = "956478653FDA85694A603C4557378A94CD73E261994E5FA5E162220C6FF5B5EA";
```

输入与 NIST 数据一一对应，用户可以相应替换。

```

if(ver_flag != PASS)
{
    printf("FAIL");
}
else
{
    printf("PASS");
}

```

输出结果：1. 通过：PASS，不通过：FAIL；对应 NIST 数据，通过：true，不通过 false；

6. RSA 库函数

6.1. 函数接口

文件：

- rn8xxx_ll_sea_rsa.c
- rn8xxx_ll_sea_rsa.h

函数名	函数接口名
公私钥对生成函数	ErrStatus LL_RSA_keygen(uRSA_t *rsa);
签名生成函数	ErrStatus LL_RSA_sign_gen(uRSA_t *rsa, const uint32_t hashData[], uint32_t hashtype, uint32_t sign[]);
签名验证函数	ErrStatus LL_RSA_sign_ver(uRSA_t *rsa, const uint32_t hashData[], uint32_t hashtype, uint32_t sign[]);
公钥加密函数	ErrStatus LL_RSA_pub_encrypt(uRSA_t *rsa, const uint32_t plain[], uint32_t plainlen, uint32_t cipher[]);
公钥解密函数	ErrStatus LL_RSA_pub_decrypt(uRSA_t *rsa, const uint32_t cipher[], uint32_t cipherlen, uint32_t plain[]);
私钥加密函数	ErrStatus LL_RSA_pvt_encrypt(uRSA_t *rsa, const uint32_t plain[], uint32_t plainlen, uint32_t cipher[]);
私钥解密函数	ErrStatus LL_RSA_pvt_decrypt(uRSA_t *rsa, const uint32_t cipher[], uint32_t cipherlen, uint32_t plain[]);

6.2. 函数说明

6.2.1. 公私钥对生成函数

ErrStatus LL_RSA_keygen(uRSA_t *rsa);

输入密钥长度，生成一对公私钥。

6.2.1.1. 参数说明

参数	说明	适用对象	注意事项
rsa	用户指定密钥长度	全部	仅支持生成 512bits 长度的公私钥对，配置 rsa_keylen=512。 指针指向地址 word 对齐；

			数据存储方式为 Little-Endian。
--	--	--	------------------------

6.2.1.2. uRSA_t 结构体说明

```
typedef struct {
    uint32_t rsa_keylen; /* key length of RSA */
    uint32_t n_len; /* bit length of modulus N */
    uint32_t e_len; /* bit length of public exponent E */
    uint32_t d_len; /* bit length of private exponent D */
    uint32_t n[32]; /* the RSA modulus */
    uint32_t e[32]; /* the RSA public exponent */
    uint32_t d[32]; /* the RSA private exponent */
} uRSA_t;
```

6.2.1.3. 返回值说明

返回值	说明
SUCCESS	生成成功
ERROR	生成失败

6.2.2. 签名生成函数

ErrStatus LL_RSA_sign_gen(uRSA_t *rsa, const uint32_t hashData[], uint32_t hashtype, uint32_t sign[]);
依照用户输入的哈希值，以及指定的密钥长度，生成签名。

6.2.2.1. 参数说明

参数	说明	适用对象	注意事项
rsa	用户指定密钥长度	全部	支持 512bits/1024bits。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
hashData	用户输入哈希结果	全部	支持 SHA256/SHA512。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
hashtype	用户输入哈希类型	全部	哈希结果的 bits 长度，比如 256/512。
sign	签名结果	全部	签名生成结果保存在该指针指向的地址。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。

6.2.2.2. 返回值说明

返回值	说明
SUCCESS	生成成功
ERROR	生成失败

6.2.3. 签名验证函数

ErrStatus LL_RSA_sign_ver(uRSA_t *rsa, const uint32_t hashData[], uint32_t hashtype, uint32_t sign[]);

依照用户输入的哈希值，以及指定的密钥长度，验证签名。

6.2.3.1. 参数说明

参数	说明	适用对象	注意事项
rsa	用户指定密钥长度	全部	支持 512bits/1024bits。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
hashData	用户输入哈希结果	全部	支持 SHA256/SHA512。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
hashtype	用户输入哈希类型	全部	哈希结果的 bits 长度，比如 256/512。
sign	用户输入的签名信息	全部	不能超过密钥长度，比如使用 RSA512 时，不能验证 1024bits 长度的签名。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。

6.2.3.2. 返回值说明

返回值	说明
SUCCESS	检验通过
ERROR	检验失败

6.2.4. 加解密函数

6.2.4.1. 公钥加密、私钥加密

公钥加密：ErrStatus LL_RSA_pub_encrypt(uRSA_t *rsa, const uint32_t plain[], uint32_t plainlen, uint32_t cipher[]);

私钥加密：ErrStatus LL_RSA_pvt_encrypt(uRSA_t *rsa, const uint32_t plain[], uint32_t plainlen, uint32_t cipher[]);

用户指定密钥信息，对明文进行加密。

6.2.4.1.1. 参数说明

参数	说明	适用对象	注意事项
rsa	用户指定密钥长度	全部	支持 512bits/1024bits。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
plain	用户输入明文	全部	指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
plainlen	用户输入明文 bits 长度	全部	长度不能超过密钥长度-11bytes。
cipher	指向密文结果地址的指针	全部	指针指向地址 word 对齐； 数据存储方式为 Little-Endian。

6.2.4.1.2. 返回值说明

返回值	说明
SUCCESS	加密成功
ERROR	加密失败

6.2.4.2. 公钥解密、私钥解密

公钥解密：ErrStatus LL_RSA_pub_decrypt(uRSA_t *rsa, const uint32_t cipher[], uint32_t cipherlen, uint32_t plain[]);

私钥解密：ErrStatus LL_RSA_pvt_decrypt(uRSA_t *rsa, const uint32_t cipher[], uint32_t cipherlen, uint32_t plain[]);

用户指定密钥信息，对密文进行解密。

6.2.4.2.1. 参数说明

参数	说明	适用对象	注意事项
rsa	用户指定密钥长度	全部	支持 512bits/1024bits。 指针指向地址 word 对齐； 数据存储方式为 Little-Endian。
cipher	用户输入密文	全部	指针指向地址 word 对齐； 数据存储方式为 Little-Endian。

cipherlen	用户输入密文 bits 长度	全部	长度不能超过密钥长度。
plain	用户输入明文	全部	指针指向地址 word 对齐； 数据存储方式为 Little-Endian。

6.2.4.2.2. 返回值说明

返回值	说明
SUCCESS	解密成功
ERROR	解密失败

6.3. 验证函数

6.3.1. 密钥生成 Demo

函数：void rsa512_keygen(void);

函数说明：指定密钥长度为 512bits，调用 LL_RSA_keygen 密钥生成函数，密钥生成函数中会自动进行一次密钥校验，所以生成函数返回值 SUCCESS 时说明密钥生成成功，且符合 RSA 标准，打印密钥模数 N，公共指数 E 和秘密指数 D。由于生成 RSA 公私钥需要生成随机数素数，建议多次调用密钥生成函数。

```
void rsa512_keygen(void) {
    int i;
    uRSA_t rsa;
    /* NOTE: generate key may fail, just try again */
    /* 1. set key length */
    rsa.rsa_keylen = 512U;
    /* 2. generate the value of key for rsa576 */
    if (SUCCESS != LL_RSA_keygen(&rsa)) {
        SEA_MSG("\n\nFAIL\n\n");
    } else {
        SEA_MSG("\n\nPASS\n\n");
    }
}
```

6.3.2. 加解密 Demo

函数：void rsa1024_demo(void);

函数说明：指定密钥长度为 1024bits，输入 N/E/D 数据，对明文进行加密得到密文，再对密文进行解密得到信息，比对信息和明文。注意调用公钥加密，则需要私钥解密；私钥加密，则需要公钥解密。

函数流程：

1. 调用加密函数 LL_RSA_pub_encrypt，输入公私钥对信息、明文和明文长度，输出密文；
2. 调用解密函数 LL_RSA_pvt_decrypt，输入公私钥对信息、密文和密文长度，输出明文；
3. 进行明文和解密后的明文比对，输出验证结果。

6.3.3. 签名和验签 Demo

函数: void rsa1024_signver(void);

函数说明: 指定密钥长度为 1024bits, 输入 N/E/D 公私钥信息, 对明文 M 进行 hash 得到哈希值 H, 调用签名函数 LL_RSA_sign_gen 对 H 进行签名得到 sign, 再调用验签函数 LL_RSA_sign_ver 对 sign 进行解密得到 H, 比对信息返回验签结果是验签成功还是失败。

6.3.3.1. 典型数据

若输入字符串数据, 则需转成 16 进制小端存储的 words 数组。

// 输入公私钥信息

```
uint32_t nn[] = {0x3E7F0445U, 0xEB866AD6U, 0x18AC4E99U, 0x7E1F9757U,  
                 0xA8386483U, 0x7E5B20A4U, 0x18DB826CU, 0xF0CC63DDU,  
                 0xB9A3C0BEU, 0x3D1EF794U, 0x0AB866A1U, 0x916EAB5AU,  
                 0xA463384FU, 0x403B5EF0U, 0xADA005F5U, 0xCF1C5042U,  
                 0x512B808AU, 0x79835714U, 0x399C3239U, 0x24FF3CFFU,  
                 0x5C1CFF61U, 0xCBC04EDAU, 0xBA60DA06U, 0xB10435C3U,  
                 0x18083478U, 0xBFC53BCFU, 0xDCFC09ACU, 0x88BE489AU,  
                 0xACD76CF7U, 0x2FAD9677U, 0x971BCEAEU, 0xC3ED70F1U};
```

```
uint32_t ee[] = {0x10001U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U,  
                 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U,  
                 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U, 0U};
```

```
uint32_t dd[] = {0x0524D4E1U, 0xB7DC1058U, 0x96603B79U, 0xE7A68E70U,  
                 0xF7205970U, 0x3643DC30U, 0x58C486C9U, 0xA6A60420U,  
                 0x2BB9F47EU, 0xF4286288U, 0xD011932DU, 0xCC8CB967U,  
                 0x822D4C6AU, 0xAFCBC348U, 0xE3069F6CU, 0x138FFCDBU,  
                 0xB3BD6A80U, 0x28A792E5U, 0xEC85CACEU, 0x76992AE3U,  
                 0xE3B72DD8U, 0xBD54B5BEU, 0x4E966EE8U, 0xDA750F12U,  
                 0xDDF0A15FU, 0xE6EBD4F4U, 0x3DE20577U, 0x3326134FU,  
                 0xC5801D9BU, 0xAD17C0E0U, 0xC40B59C6U, 0x78137C25U};
```

...

// 对明文进行哈希计算得到哈希结果并进行处理, 这里选用 SHA-256

```
hash_gen(message256, &sha256_cfg, Hm, Curve_NIST_P256.num_bytes);
```

```
uint32_t cm[32] = {0};
```

```
for (i = 7; i >= 0; i--) {
```

```
    cm[i] = Hm[i];
```

```
}
```

6.3.3.2. 测试流程

// 签名

```
uint32_t sign[32] = {0};
```

```
LL_RSA_sign_gen(&rsa, cm, 256, sign);  
...  
// 验签  
passflag = LL_RSA_sign_ver(&rsa, cm, 256, sign);  
...
```

7. 其他功能

7.1. 可选宏定义

文件:

- rn8xxx_ll_sea_def.h

7.1.1. AES 地址对齐方式定义

作用对象: 明文、密文、附件消息 buffer 地址对齐方式配置。默认为 U8 地址对齐。可以配置成 U32 地址对齐, 提高软件效率。

配置操作:

- U8 地址对齐 (默认): 使能宏定义 `#define AES_CRYPT_ITEMS_IN_BYTES`
- U32 地址对齐: 注释掉或者删除宏定义 `/*#define AES_CRYPT_ITEMS_IN_BYTES*/`

7.1.2. HASH 地址对齐方式定义

作用对象: 消息输入和结果输出地址对齐方式配置。默认为 U8 地址对齐。可以配置成 U32 地址对齐, 提高软件效率。

配置操作:

- U8 地址对齐 (默认): 使能宏定义 `#define HASH_CRYPT_ITEMS_IN_BYTES`
- U32 地址对齐: 注释掉或者删除宏定义 `/*#define HASH_CRYPT_ITEMS_IN_BYTES*/`

7.1.3. ECC 地址对齐方式定义

作用对象: 消息输入和结果输出地址对齐方式配置。默认为 U8 地址对齐。可以配置成 U32 地址对齐, 提高软件效率。

配置操作:

- U8 地址对齐 (默认): 使能宏定义 `#define ECC_CRYPT_ITEMS_IN_BYTES`
- U32 地址对齐: 注释掉或者删除宏定义 `/*#define ECC_CRYPT_ITEMS_IN_BYTES*/`